

Kubernetes · с нуля, руками, до собеса

Kubernetes — лабораторная тетрадь.

Поднимаешь свой кластер через kubespray на 3 VM (cri-o + Calico), щупаешь сети, RBAC, etcd, разбираешь `kubectl apply` по шагам. Каждую команду копируешь и выполняешь руками — на собесе говоришь то, что реально делал.

3 VM → свой кластер

cri-o + Calico VXLAN

копируемые команды + вывод

debug-секции

AI-запросы с проверкой



Исходники, манифесты и приложение — на GitHub
github.com/boost-mentor/ultimate-devops-guide

t.me/boostmentor



Виктор Шутов

DevOps-инженер · ментор

boostmentor.ru →

Открыл → скопировал → повторил → понял

Инструкция, по которой ты поднимаешь кластер и трогаешь каждую тему руками. Команды крупные, широкие, копируемые. У каждого блока метка: где вводить и что должно получиться.

Метки «где вводить»

LOCAL

твоя машина (mac/linux)

NODE1

control-plane нода

POD

внутри пода

CLUSTER

любой узел с kubectl

Структура каждой практической темы

- 1 **Задача** — что и зачем делаем.
- 2 **Команды** — широкий терминал, реально копируется, с комментариями.
- 3 **Ожидаемый вывод** — что должно появиться, и как его читать.
- 4 **Если сломалось** — чем диагностировать.
- 5 **Собес-вывод** — как сформулировать на интервью.

! Заглушки в командах

Везде, где видишь `<NODE1_IP>`, `<NODE1_PRIVATE_IP>` — подставь свои адреса. Реальные IP/пароли в тетрадь не зашиты намеренно.



СПРОСИ AI ПРАВИЛЬНО — МЕТОД

Я изучаю Kubernetes как DevOps-инженер уровня junior→middle.
Объясняй коротко, без «магии». После объяснения — НЕ простыня, а:

- ① один уточняющий вопрос мне на проверку понимания;
- ② команды, которыми я докажу это на СВОЁМ кластере.

↳ **проверь:** этот шаблон подставляй в начало любого запроса ниже по тетради.



Что внутри

Открыл раздел → повторил команды на своём кластере. Слева — страница, где раздел начинается.

СТР	РАЗДЕЛ	ЧТО ВНУТРИ
4	База — Docker и сети	контейнер≠ВМ · build/run · Alpine · veth · DNS · ping=ICMP
13	Доступ по SSH	ключ vs пароль · ssh-config · ssh-agent · /etc/hosts · debug -vvv
16	Раскатка своего кластера	kubespray: SSH → inventory → playbook → kubeconfig → Calico → cross-node
24	Кластер	архитектура · kubectl · умер под · доступ · RBAC · etcd живьём
34	Сущности	координаты · Pod · sidecar · сеть пода · Deployment · Service · Config · Storage
49	Финал	kubectl apply по шагам · пантеон · итог + что дальше
53	Под капотом	kube-proxy iptables · VXLAN по байтам · Raft · scheduler · OCI · cgroups
67	Справочник манифестов	готовый YAML каждой сущности + разбор полей
84	Troubleshooting	7 плейбуков: Pending · ImagePull · CrashLoop · Service · DNS · NotReady
96	Собес-тренажёр	28 вопросов с ловушками по грейдам
102	Практика	8 заданий с промтами для ИИ
104	Командная шпаргалка	все команды по темам: kubectl · crictl · etcdctl · ansible
107	AI-запросы	как просить нейронку по темам + проверка
108	BoostMentor · что дальше	Telegram-канал · YouTube · менторство

Kubernetes = система управления желаемым состоянием

Контейнеры решили задачу «упаковать приложение с зависимостями». Но когда контейнеров много и машин много — управлять руками больно. Kubernetes — это **оркестратор контейнеров**: запускает их на кластере машин, перезапускает упавшие, масштабирует, обновляет без даунтайма, даёт сеть и service discovery. Под всем этим одна идея: ты описываешь **желаемое состояние** («хочу 3 реплики такого образа, такие лимиты»), а куб постоянно приводит реальность к этому описанию.

Когда куб НЕ нужен, а когда нужен

КУБ НЕ НУЖЕН — ЛИШНИЙ OVERHEAD	КУБ НУЖЕН — ОН ОКУПАЕТСЯ
стартап / пилот / MVP	много сервисов, которые надо связать
один сервис на одной машине	много машин, на которые надо размазать нагрузку
локальная разработка → <code>docker compose</code>	нужен scaling (горизонтально, реплики)
команды и людей под инфру нет	нужен self-heal (упал под — поднимется новый)
трафик стабильный, без всплесков	нужен rolling update + service discovery

i Мета-правило: чем больше всего этого надо — тем нужнее куб

Docker Compose — это одна машина, удобно для локалки и MVP (поднять backend+postgres+redis рядом). Docker Swarm был, но де-факто стандартом стал Kubernetes. Если надо мало (один сервис, одна машина) — куб только мешает: лишняя инфраструктура, люди, сложность.

✓ Собес-приём: минимальный ответ → углубляй по запросу

На вопрос «что такое Kubernetes» сначала дай **минимальный корректный ответ** — «оркестратор контейнеров, система управления желаемым состоянием». НЕ вываливай сразу etcd / CNI / RBAC. Спроси «рассказать подробнее?» и углубляйся по грейду. Глубина ответа должна соответствовать вопросу, а не топить интервьюера.

СОБЕС

«Kubernetes — оркестратор контейнеров: управляет ими на кластере машин, держит **желаемое состояние**. Нужен, когда много сервисов и машин, нужны scaling / self-healing / rolling update / service discovery. Для одного сервиса на одной машине это лишний overhead — там хватит docker compose».



Контейнер ≠ VM: это процесс хоста с изоляцией

Задача: доказать, что контейнер — это НЕ «маленькая виртуалка с отдельной ОС», а обычный процесс хостового ядра, которому через **namespaces** урезали видимость, а через **cgroups** — ресурсы. Запускаем nginx и лезем в его кишки прямо с ноды.

NODE контейнер = процесс хоста: namespaces + cgroups

```
docker run -d --name demo nginx # подняли контейнер в фоне

# PID процесса контейнера НА ХОСТЕ (он же просто процесс ядра хоста)
PID=$(docker inspect -f '{{.State.Pid}}' demo)
echo $PID

# namespaces — ЧТО контейнер видит (свои PID/сеть/mount/...)
ls -l /proc/$PID/ns # net, pid, mnt, uts, ipc, user
nsenter -t $PID -n ip addr # заходим в его net-namespaces

# cgroups — СКОЛЬКО ресурсов контейнер может съесть
cat /proc/$PID/cgroup # к какой cgroup привязан процесс
docker stats --no-stream demo # текущие CPU / MEM лимиты и расход
```

ОЖИДАЕМЫЙ ВЫВОД

```
lrwxrwxrwx 1 root root 0 net → 'net:[4026532...]' # ← свой сетевой namespace
lrwxrwxrwx 1 root root 0 pid → 'pid:[4026532...]' # ← свой PID namespace
...
1: eth0: ... inet 172.17.0.2/16 # сеть, которую видит ТОЛЬКО контейнер

CONTAINER CPU % MEM USAGE / LIMIT
demo 0.00% 2.5MiB / 7.7GiB # ← cgroups считают/лимитируют ресурсы
```

Читаем: `/proc/$PID/ns` существует на ХОСТЕ — значит процесс контейнера живёт в ядре хоста, у него просто свои namespaces. У VM так не выйдет: там отдельная гостевая ОС со своим ядром, и с хоста в её `/proc` ты не залезешь.

i Namespaces vs cgroups vs ядро

namespaces = ЧТО процесс видит (свои PID, сеть, точки монтирования, hostname). **cgroups** = СКОЛЬКО ресурсов может съесть (CPU / RAM / IO). **Ядро — общее с хостом:** контейнер его не поднимает, в отличие от VM. Поэтому контейнеры легче, быстрее стартуют и плотнее упаковываются — но изоляция мягче, чем у VM с отдельной ОС.

На нашей cri-o-ноде Docker нет — то же через crictl: `crictl ps` → `crictl inspect <CID> | grep pid` (взять PID) → дальше те же `ls /proc/<PID>/ns`, `cat /proc/<PID>/cgroup`, `crictl stats`. Docker-вариант — как канонический пример «как это устроено».

СОБЕС

«Контейнер — не виртуалка, а процесс хостового ядра. Изоляция = **namespaces** (что процесс видит) + **cgroups** (сколько ресурсов может съесть), ядро общее с хостом. Проверяю руками: беру PID процесса, смотрю `/proc/PID/ns` и `cgroup`. VM же поднимает целую гостевую ОС со своим ядром — отсюда тяжелее, но изоляция жёстче».



Docker: build ≠ run (образ ≠ контейнер)

Задача: разнести два действия, которые новички путают. **build** собирает **ОБРАЗ** из Dockerfile. **run** запускает из образа **КОНТЕЙНЕР** (run = create + start, собирать НЕ умеет). Собираем своё приложение, запускаем, проверяем, что это живой изолированный процесс.

LOCAL build → images → run → проверка

```
# 1) BUILD: собираем ОБРАЗ из Dockerfile ('.' = папка, где лежит Dockerfile)
docker build -t app:1.0.0 .

# 2) образ появился в локальном реестре — смотрим размер
docker images app

# 3) RUN: из образа запускаем КОНТЕЙНЕР (-d фон, -p проброс порта, --rm убрать после)
docker run --rm -d -p 8080:8080 --name good app:1.0.0

# 4) приложение отвечает — значит изолированный процесс реально работает
curl localhost:8080/quote
curl -s localhost:8080/healthz # health-эндпоинт

# 5) заходим ВНУТРЬ контейнера (Alpine → есть sh)
docker exec -it good sh
```

ОЖИДАЕМЫЙ ВЫВОД

REPOSITORY	TAG	IMAGE ID	SIZE	
app	1.0.0	a1b2c3d4e5f6	18MB	# ← образ собрался
{ "quote": "We are Sex Bob-omb!" }				# /quote — приложение живо
ok				# /healthz
/app #				# мы внутри контейнера

Читаем: **docker images** показал образ — это результат **build**. **curl** ответил — это результат **run**. Образ можно собрать один раз и запустить из него сколько угодно контейнеров; контейнер без образа не существует.

i build = образ, run = create + start

docker build читает Dockerfile и печёт неизменяемый **образ** (шаблон). **docker run** берёт образ, создаёт из него контейнер (create) и запускает (start). **run собирать не умеет** — если образа нет, он сначала попытается его **pull** из реестра, а не соберёт из Dockerfile. Этот же образ потом катим в k8s.

СОБЕС

«**build** печёт образ из Dockerfile — неизменяемый шаблон. **run** = create+start контейнера из образа; собирать он не умеет, максимум стянет образ из реестра. Один образ → сколько угодно контейнеров. Контейнер без образа не существует».



Плохой и хороший образ: ~302MB → ~18MB (×17)

Задача: взять ОДИН и тот же Go-код и показать, как упаковка решает всё. Наивный одностадийный Dockerfile тащит внутрь весь Go-тулчейн (~302MB). Multistage + Alpine оставляет только бинарь на крошечной базе (~18MB). Код тот же — отличается только упаковка.

LOCAL Dockerfile.simple — наивный, 1 стадия, ~302MB

```
FROM golang:1.25-alpine          # весь Go-тулчейн уезжает в финальный образ
WORKDIR /app
COPY . .                        # кэш зависимостей не используем — всё одним слоем
RUN go build -o app .           # компилятор, кэш, исходники — всё внутри
CMD ["/app"]                   # итог ~302MB: 95% — мусор, который в проде не нужен
```

LOCAL Dockerfile — multistage + Alpine, ~18MB

```
# — СТЕЙДЖ 1: builder — тут собираем, в финал НЕ попадёт —
FROM golang:1.25-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./          # сначала ТОЛЬКО манифесты зависимостей
RUN go mod download            # ← отдельный слой: кэшируется, пока go.mod не менялся
COPY . .                      # теперь код (меняется часто — не сбивает кэш выше)
RUN CGO_ENABLED=0 go build -o app . # статичный бинарь, без зависимости от libc

# — СТЕЙДЖ 2: финальный — только бинарь на чистом Alpine —
FROM alpine:3.20
RUN adduser -D -u 10001 appuser # не-root юзер для безопасности
COPY --from=builder /app/app /app/app # ← тянем ТОЛЬКО артефакт из builder
USER appuser                    # контейнер бежит не от root
ENTRYPOINT ["/app/app"]        # итог ~18MB: только то, что нужно в рантайме
```

Читаем: `go.mod` + `go mod download` идут ДО `COPY . .` — поэтому слой с зависимостями кэшируется и не пересобирается при каждой правке кода. Финальный `FROM alpine` начинает образ с нуля, и в него `COPY --from=builder` кладёт ТОЛЬКО бинарь — тулчейн остаётся в выброшенном builder-стейдже.

⚠ Слой ≠ стейдж — частый вопрос

Слой = одна инструкция `RUN` / `COPY` / `ADD` (единица кэша). **Стейдж** = блок, начинающийся с `FROM`. В multistage стейджей несколько, но в финальный образ попадает только последний `FROM` + то, что в него явно скопировали через `--from`. Всё остальное (builder с компилятором) выбрасывается.

СОБЕС

«Multistage: в builder-стейдже собираю бинарь, в финальный `FROM alpine` тащу `--from=builder` только артефакт — тулчейн не уезжает в прод. `go mod download` ставлю ДО копирования кода, чтобы кэшировать зависимости. Бьюсь за маленький образ: быстрее pull, меньше CVE-поверхность. Слой — это инструкция, стейдж — это FROM».



Alpine vs distroless + под какую архитектуру

Задача: выбрать базовый образ осознанно и не словить `exec format error` из-за архитектуры. Образ собирается под **платформу = ОС + архитектура CPU**, и она обязана совпасть с нодой.

Alpine vs distroless

КРИТЕРИЙ	ALPINE	DISTROLESS
размер	~5-18MB	ещё меньше
shell внутри	есть (<code>sh</code>) → можно зайти	НЕТ shell
дебаг	<code>kubectl exec -- sh</code>	только ephemeral debug-контейнер
CVE-поверхность	малая	минимальная (безопаснее)
когда берём	учимся / надо лазить внутрь	прод, максимум безопасности

Почему мы берём Alpine: чтобы можно было зайти внутрь и показывать (`kubectl exec -- sh`). Distroless меньше и безопаснее, но shell там нет — дебажить придётся ephemeral-контейнером.

CLUSTER дебаг контейнера БЕЗ своего shell — netshoot рядом

```
# подключаем временный контейнер с инструментами в тот же под/неймспейсы
kubectl debug -it <pod> \
  --image=nicolaka/netshoot \
  --target=<container> # делит namespaces с целевым контейнером

# внутри netshoot есть ip / dig / curl / ss — а в distroless-аппе их нет

ОЖИДАЕМЫЙ ВЫВОД
Targeting container "app". ...
Defaulting debug container name to debugger-xxxxx.
<container> bash-5.2# # shell ЕСТЬ — но это netshoot, не сам distroless
```

⚠ Образ = ОС + архитектура: amd64 ≠ arm64

amd64 (x86_64 — Intel/AMD, большинство серверов) vs **arm64** (ARM — Apple Silicon M1/M2, AWS Graviton, Raspberry Pi). ОС у контейнеров — **Linux** (используют ядро Linux; «macOS-контейнеров» нет — на Mac Docker крутит Linux-VM, образ всё равно `linux/<arch>`). Собрал на Mac (arm64), а ноды amd64 → `exec format error`, под не стартует.

LOCAL собрать под нужную архитектуру ноды

```
# 1) явно указать платформу при сборке
docker build --platform linux/amd64 -t app:1.0.0 .

# 2) для Go — кросс-компиляция прямо в Dockerfile (builder-стейдж)
CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -o app .

# 3) сразу multi-arch — нода сама тянет свой вариант:
# docker buildx build --platform linux/amd64,linux/arm64 ... --push
```

🔊 Собес-ловушка: как дебажить distroless без shell?

В distroless нет `sh` — `kubectl exec -- sh` упадёт. Дебажат **ephemeral debug-контейнером**:

`kubectl debug -it <pod> --image=nicolaka/netshoot --target=<container>` — он подключается в namespaces целевого контейнера и приносит свои инструменты, не меняя сам образ приложения.

СОБЕС

«Беру Alpine, когда надо лазить внутрь; distroless — для прода, он меньше и безопаснее, но без shell, поэтому дебаг через ephemeral debug-контейнер (`kubectl debug --image=netshoot --target`). Образ собирается под платформу = ОС + арх; архитектура обязана совпасть с нодой, иначе `exec format error`. Чиню это `--platform` / `GOOS GOARCH` / multi-arch buildx».

Docker bridge — идеальная база сети

Прежде чем лезть в Calico, разбери сеть на Docker — она проще, но устроена ровно так же. Запустил контейнер — у него свой netns, свой `eth0`, свой IP. Но `eth0` — не физкарта, а один конец **veth-пары**; второй конец воткнут на хосте в Linux-bridge `docker0` (виртуальный свитч).

Задача: поднять nginx, посмотреть его сеть изнутри и найти второй конец «провода» на хосте. Делаем на **Linux-хосте с Docker** (не на cri-o-ноде, не на Mac).

NODE-DOCKER сеть контейнера изнутри + второй конец на хосте

```
# 1) поднимаем контейнер — попадает в дефолтную сеть bridge
docker run -d --name nginx-demo nginx

# 2) смотрим сеть ИЗНУТРИ контейнера
docker exec nginx-demo ip addr          # eth0 + IP из 172.17.0.0/16
docker exec nginx-demo ip route         # default via 172.17.0.1 (это docker0)

# 3) теперь НА ХОСТЕ — тот самый свитч и провода
ip addr show docker0                   # bridge-интерфейс, gateway 172.17.0.1
ip link | grep veth                    # vethXXXX@if.. — второй конец пары к контейнеру
sudo iptables -t nat -L | head          # MASQUERADE — NAT наружу
```

ОЖИДАЕМЫЙ ВЫВОД (ФРАГМЕНТЫ)

```
eth0@if7: ... inet 172.17.0.2/16      # внутри контейнера
docker0: ... inet 172.17.0.1/16      # на хосте — свитч=шлюз
veth1a2b3c@if6: ... master docker0  # провод воткнут в docker0
MASQUERADE all -- 172.17.0.0/16 !172.17.0.0/16 # NAT наружу
```

Читаем вывод: IP контейнера `172.17.0.2`, шлюз `172.17.0.1` = `docker0`. Номер за `@if` у `eth0` внутри = индекс `veth` на хосте — так находишь, какой провод к какому контейнеру.

4 сетевых драйвера Docker

bridge — дефолт (свой netns + `docker0`); **host** (`--network host`) — сеть хоста напрямую, БЕЗ изоляции; **none** — только `lo`, наружу никак; **user-defined bridge** — как bridge, но добавляется **DNS по имени контейнера**. Наружу пакет идёт через **NAT** (iptables `MASQUERADE`), внутрь — `docker run -p 8080:80` кладёт правило **DNAT** (`host:8080 → container:80`).

✓ veth pair = патч-корд

Это пара виртуальных устройств, соединённых как патч-корд: что влетело в один конец — вылетает из другого. Один конец = `eth0` внутри контейнера, второй = `vethXXXX` на хосте в bridge. В k8s ровно то же: `eth0` пода ↔ `cali...` на нодe.

Сеть compose: DNS по имени + путь пакета

Поднимаем своё приложение + `postgres` + `netshoot` (сетевой зонд — в Alpine-аппе нет `ip` / `dig`) в одной сети. Compose делает **user-defined bridge** (это НЕ `docker0`), а значит включается DNS: до соседа ходим по **имени сервиса**, не по IP.

Задача: доказать, что имя `postgres` резолвится через Docker-DNS, и проследить путь пакета — до соседа напрямую, наружу через шлюз.

NETSHOOT DNS по имени и куда уходит пакет

```
# поднимаем стенд и заходим в зонд
docker compose up -d
docker compose exec netshoot bash

# --- дальше ВНУТРИ netshoot ---
ip addr                                # eth0 = свой конец veth-пары
ip route                               # локальное → eth0, остальное → gateway
cat /etc/resolv.conf                  # nameserver 127.0.0.11 = Docker DNS (НЕ /etc/hosts)

getent hosts postgres                 # имя сервиса → IP (это и есть DNS)

# путь пакета — не магия, а route lookup:
ip route get $(getent hosts postgres | awk '{print $1}') # до postgres — НАПРЯМУЮ через
ip route get 8.8.8.8                    # наружу — ЧЕРЕЗ gateway
```

ОЖИДАЕМЫЙ ВЫВОД (КЛЮЧЕВЫЕ СТРОКИ)

```
nameserver 127.0.0.11                # встроенный Docker DNS
postgres 172.20.0.3                  # имя → IP
172.20.0.3 dev eth0 src 172.20.0.4    # сосед: та же подсеть, БЕЗ gateway
8.8.8.8 via 172.20.0.1 dev eth0      # наружу: через gateway (bridge)
```

Читаем вывод: до `postgres` Linux выбрал маршрут **напрямую через `eth0`** — он в той же подсети, шлюз не нужен. До `8.8.8.8` — **через gateway** (это bridge на хосте, дальше NAT в интернет). Имя сервиса превратилось в IP именно через DNS `127.0.0.11`, а не через `/etc/hosts`.

i Путь пакета целиком (запомни цепочку)

имя → DNS (127.0.0.11) → IP → **route lookup** → `eth0` (veth) → **bridge** → veth → `postgres`. Магии нет: на каждом шаге — обычная таблица маршрутов и интерфейсы. Это и есть прообраз пути запроса в k8s.

СОБЕС

«Docker — ранняя модель k8s: **service→DNS, container IP→сеть, route→путь, eth0/veth→выход, bridge→свитч**. В k8s то же самое, только между нодами добавляется CNI (Calico/VXLAN). Путь пакета не угадываю — показываю через `getent` + `ip route get`».

Собес-ловушка: «на каком порту работает ping?»

Раз уж пингуем соседа — типичная подколка с собес. Ответ ломает половину кандидатов, потому что путают уровни модели.

🔊 «На каком порту работает ping?» → НИ НА КАКОМ

У **ping портов нет**. ping = **ICMP** — это **L3**, рядом с IP: есть адреса, портов нет. ICMP Echo Request упакован прямо в IP-пакет (**protocol = ICMP**, поля **dst port** попросту не существует). **Порты только на L4** — TCP/UDP (80/443/5432/22). IP всё равно нужен: **ping google.com** → DNS (имя→IP) → ICMP на IP. **Главный вывод: ping успешен ≠ сервис работает** — хост лишь ответил на ICMP; а порт может быть закрыт. И наоборот: ICMP бывает закрыт firewall'ом — ping не идёт, а сервис на порту жив. Порт проверяй ОТДЕЛЬНО.

Задача: увидеть руками разницу между «хост отвечает» (ICMP) и «сервис на порту жив» (TCP).

CLUSTER три РАЗНЫХ проверки — три разных уровня

```
ping -c2 host           # L3/ICMP: хост вообще живой? (портов тут нет)
nc -vz host 443         # L4/TCP: открыт ли КОНКРЕТНЫЙ порт
curl -v http://host     # L7: реально ли ОТВЕЧАЕТ приложение (HTTP)
```

ОЖИДАЕМЫЙ ВЫВОД (КАК ЧИТАТЬ)

```
2 packets transmitted, 2 received      # ICMP ок — но это НЕ про сервис
Connection to host 443 port [tcp] succeeded! # порт 443 открыт
< HTTP/1.1 200 OK                      # приложение реально ответило
```

Читаем вывод: **ping** прошёл, но это говорит только «хост в сети». Жив ли сервис — отвечает **nc -vz** (порт) и **curl -v** (приложение). Три команды — три слоя, не смешивай их в один ответ.

Кто за что отвечает — по слоям

АДРЕС	ДО КОГО	ЕСТЬ ЛИ ПОРТ
MAC	до следующего hop (L2, сосед по сети)	нет
IP	до конечной цели (L3, сквозь сеть)	нет
порт	до приложения на хосте (L4)	только TCP/UDP

СОБЕС «ping — это ICMP на L3, у него нет портов; порты живут на L4 (TCP/UDP). Поэтому **ping успешен ≠ сервис работает**: проверяю порт отдельно через **nc -vz** или **curl -v**. MAC ведёт до соседнего hop, IP — до конечной цели, порт — только для TCP/UDP-приложения».



SSH: ключ вместо пароля

Чтобы показать Docker или зайти на ноды кластера — надо попасть на сервер. По паролю можно, но не делаем: разберём, почему ключ безопаснее, и как его правильно подцепить.

🔑 Пароль

Хранится (хешем) **на сервере**. Сервер в интернете → боты долбят брутфорсом. Набираешь каждый раз (подсмотрят / кейлоггер). Можно слить — секрет на стороне сервера.

✓ Ключ

Приватный **никогда не покидает твою машину**. На сервере только публичный (он НЕ секретный — пусть утекает). Владение доказываешь challenge-response. Брутфорсить нереально.

Суть: **публичный ключ кладётся на сервер** (`~/.ssh/authorized_keys`), **приватный держишь у себя**, заходишь без пароля. В проде пароль вообще выключают: `PasswordAuthentication no` в `sshd_config`.

Генерим пару и кладём на сервер

LOCAL ключ с кастомным именем → на сервер

```
# 1) генерим пару с ПОНЯТНЫМ именем (не дефолтное id_ed25519)
ssh-keygen -t ed25519 -f ~/.ssh/bmkey -C "boostmentor"

# 2) публичный ключ → в authorized_keys сервера (1 раз спросит пароль)
ssh-copy-id -i ~/.ssh/bmkey.pub root@<СЕРВЕР_IP>

# 3) пробуем зайти...
ssh root@<СЕРВЕР_IP>

А ОН ВСЁ ЕЩЁ ПРОСИТ ПАРОЛЬ!
root@<СЕРВЕР_IP>'s password: _
```

⚠ Почему ключ не сработал автоматом

SSH сам пробует только ключи с **дефолтными именами** (`id_ed25519` / `id_rsa` / `id_ecdsa`). Наш `bmkey` — нестандартное имя, и SSH его НЕ перебирает. Порядок источников ключа: `-i` / `IdentityFile` → `ssh-agent` → дефолтные имена. Значит ключ надо **указать явно** (следующая страница).

🔊 Сигнал на собесе

Если SSH **предложил ввести пароль** — значит ключ НЕ сработал (откат на password). Проверить только ключ, без отката:

```
ssh -o PreferredAuthentications=publickey -o PasswordAuthentication=no root@<СЕРВЕР_IP> —  
упадёт сразу, если ключ не принят.
```



Указываем ключ: config, agent и /etc/hosts

Задача: заставить SSH использовать наш `bmkey`. Два способа — оба полезны, покажем оба.

Способ 1 — ssh-config (алиас + ключ навсегда)

LOCAL ~/.ssh/config

```
Host bm                                # короткий алиас
  HostName <СЕРВЕР_IP>
  User root
  IdentityFile ~/.ssh/bmkey           # какой ключ использовать

# теперь просто:
ssh bm                                # по ключу + по алиасу, без -i и без IP
```

Когда нужен: серверов много, нестандартный порт/юзер/ключ. Работает **только** для ssh/scp.

Способ 2 — ssh-agent (ключ в памяти на сессию)

LOCAL ssh-add

```
ssh-add ~/.ssh/bmkey                 # положили приватный ключ в агент
ssh-add -l                           # проверить, что он там
ssh root@<СЕРВЕР_IP>                 # теперь идёт по ключу, без -i
```

/etc/hosts — ходим по именам (а не по IP)

LOCAL локальное имя → IP (для ВСЕХ команд)

```
sudo tee -a /etc/hosts >/dev/null <<'EOF'
<NODE1_IP> node1
<NODE2_IP> node2
<NODE3_IP> node3
EOF

ssh root@node1                       # теперь по имени работают ssh, curl, ping — всё
ping -c2 node2
```

и ssh-config ≠ /etc/hosts

ssh-config — алиас с ключом/юзером, только для ssh/scp. **/etc/hosts** — просто имя→IP на ТВОЕЙ машине (проверяется ДО DNS), работает для ВСЕХ команд (ssh/curl/ping), но без ключей. Минус /etc/hosts: поменялся IP — правь руками.

Не пускает? Дебаг

LOCAL `ssh -vvv` показывает весь хендшейк

```
ssh -vvv root@<СЕРВЕР_IP> 2>&1 | egrep 'Offering|accepts|Authenticated'
```

ЧИТАЕМ

```
Offering public key: ~/.ssh/bmkey      # предложил наш ключ
Server accepts key: ...                # сервер принял
Authenticated to ... (publickey)      # вошли по ключу — успех
```

СОБЕС

«Приватный ключ у меня, публичный на сервере. Кастомное имя ключа SSH сам не подхватит — указываю через `IdentityFile` в `ssh-config` или через `ssh-agent`. Если сервер предложил пароль — значит ключ не приняли, гоняю `ssh -vvv`».



Свой Kubernetes через kubespray: что поднимаем

minikube — это один узел «для дома». Здесь по-взрослому: **3 чистые VM**, и с локальной машины одним прогоном ansible раскатываем настоящий кластер. Это и есть kubespray — набор плейбуков. Открыл тетрадь — повторил — поднял.

Что в итоге будет

- **node1, node2** — control-plane + etcd
- **node3** — worker (+ etcd, чтобы кворум был на 3 — нечётное число)
- Ubuntu 24.04 · container runtime **cri-o** · CNI **Calico** (VXLAN по умолчанию)
- доступ по **SSH-ключу**, ansible запускается **с локальной машины**

Схема стенда

HOSTNAME	PUBLIC IP	PRIVATE IP	РОЛЬ	SSH	ЧТО НА НОДЕ
node1	<NODE1_IP>	<NODE1_PRIVATE_IP>	control-plane + etcd	root	api-server, etcd, scheduler, controller-manager
node2	<NODE2_IP>	<NODE2_PRIVATE_IP>	control-plane + etcd	root	2-й control-plane (HA), etcd
node3	<NODE3_IP>	<NODE3_PRIVATE_IP>	worker + etcd	root	kubelet, cri-o, поды приложений, 3-й etcd

Минимум на ноду

2 CPU / 2–4 GB RAM / 20+ GB диска, чистая Ubuntu 24.04, сеть между нодами по приватным IP. Меньше 2 GB — kubelet и etcd начнут падать по памяти.

СОБЕС

«minikube — один узел для локалки; для настоящего стенда беру 3 VM и раскатываю kubespray-плейбуками: control-plane на двух нодах для HA, etcd на трёх для кворума».



Готовим локальную машину и SSH ко всем VM

Задача: проверить инструменты на своей машине и раздать единый SSH-ключ на все три ноды, чтобы ansible ходил по ключу, а не по паролю.

LOCAL проверяем инструменты + рабочая папка

```
mkdir -p ~/labs/kubespray-demo
cd ~/labs/kubespray-demo

python3 --version          # нужен Python 3.10+
ssh -V                     # OpenSSH есть в любой ОС
ansible --version || true  # если пусто – поставим ниже

# если ansible/git/python нет (macOS):
brew install python git
```

LOCAL генерим ключ и раздаём на 3 ноды

```
# 1) пара ключей с понятным именем (не дефолтное id_ed25519)
ssh-keygen -t ed25519 -f ~/.ssh/bm_k8s -C "boostmentor-k8s"

# 2) кладём ПУБЛИЧНЫЙ ключ на каждую ноду (1 раз спросит пароль)
ssh-copy-id -i ~/.ssh/bm_k8s.pub root@<NODE1_IP>
ssh-copy-id -i ~/.ssh/bm_k8s.pub root@<NODE2_IP>
ssh-copy-id -i ~/.ssh/bm_k8s.pub root@<NODE3_IP>

# 3) проверяем вход по ключу – пароль больше спрашивать не должен
ssh -i ~/.ssh/bm_k8s root@<NODE1_IP> hostname
ssh -i ~/.ssh/bm_k8s root@<NODE2_IP> hostname
ssh -i ~/.ssh/bm_k8s root@<NODE3_IP> hostname
```

ОЖИДАЕМЫЙ ВЫВОД

```
node1
node2
node3
```

Читаем вывод: три hostname без запроса пароля = ключ принят на всех нодах, ansible сможет зайти. Если просит пароль – ключ не подхватился (проверь `ssh -vvv`, строку `Server accepts key`).

⚠ Приватный ключ – только у тебя

На ноды уезжает `.pub` (публичный, не секретный). Приватный `~/ssh/bm_k8s` никому не отдаём – им ansible и доказывает, что это ты.



Имена нод и клонируем kubespray

Задача: ходить по именам (node1/2/3), задать hostname на нодах и поставить kubespray в изолированное окружение.

LOCAL /etc/hosts — ходим по именам

```
sudo tee -a /etc/hosts >/dev/null <<'EOF'
<NODE1_IP> node1
<NODE2_IP> node2
<NODE3_IP> node3
EOF

ping -c 2 node1
ping -c 2 node2
ping -c 2 node3
```

NODE1-2-3 задаём hostname (на КАЖДОЙ ноде свой)

```
# node1: hostnamectl set-hostname node1
# node2: hostnamectl set-hostname node2
# node3: hostnamectl set-hostname node3
```

LOCAL клонируем kubespray в venv

```
git clone https://github.com/kubernetes-sigs/kubespray.git
cd kubespray

python3 -m venv .venv
source .venv/bin/activate

pip install -U pip
pip install -r requirements.txt # ansible нужной версии встанет сюда
```

ОЖИДАЕМЫЙ ВЫВОД

```
Successfully installed ansible-... jinja2-... netaddr-...
```

Зачем venv: kubespray требователен к версии ansible. Изолированное окружение `.venv` не ломает системный ansible. Перед каждым запуском — `source .venv/bin/activate`.



Inventory: описываем ноды и роли

Задача: сказать kubespray, какие ноды есть и кто из них control-plane / worker / etcd. Берём шаблон `sample` и правим под себя.

LOCAL копируем шаблон `inventory`

```
# из папки kubespray, .venv активирован
cp -rfp inventory/sample inventory/bm-cluster
```

LOCAL `inventory/bm-cluster/hosts.yaml`

```
all:
  hosts:
    node1: { ansible_host: <NODE1_IP>, ip: <NODE1_PRIVATE_IP>, access_ip: <NODE1_PRIVATE_IP> }
    node2: { ansible_host: <NODE2_IP>, ip: <NODE2_PRIVATE_IP>, access_ip: <NODE2_PRIVATE_IP> }
    node3: { ansible_host: <NODE3_IP>, ip: <NODE3_PRIVATE_IP>, access_ip: <NODE3_PRIVATE_IP> }
  children:
    kube_control_plane:      # мозг кластера — на node1, node2
      hosts: { node1: {}, node2: {} }
    kube_node:               # где крутятся поды
      hosts: { node1: {}, node2: {}, node3: {} }
    etcd:                    # память кластера — 3 ноды = кворум
      hosts: { node1: {}, node2: {}, node3: {} }
    k8s_cluster:
      children: { kube_control_plane: {}, kube_node: {} }
    calico_rr: { hosts: {} }
```

Логика: `access_ip` = по какому адресу ноды общаются между собой (приватная сеть). `etcd` на 3 нодах, потому что кворум = $N/2+1$ — три ноды переживут падение одной.



Runtime + сеть, и запускаем playbook

Задача: выбрать container runtime (cri-o) и CNI (Calico), затем одним прогоном ansible раскатать кластер.

LOCAL inventory/bm-cluster/group_vars/...

```
# k8s_cluster/k8s-cluster.yml:
container_manager: cri-o
kube_network_plugin: calico
```

⚠ Имена переменных — сверь с версией

kubespray меняет имена переменных между релизами. Точные имена смотри в

[inventory/sample/group_vars/](#) своей версии, а не пиши наугад. Это частая причина «плейбук молча не применил настройку».

LOCAL проверка связи → раскатка

```
source .venv/bin/activate

# 1) ansible достучался до всех нод по ключу?
ansible all -i inventory/bm-cluster/hosts.yaml \
  -m ping --private-key ~/.ssh/bm_k8s -u root

# 2) РАСКАТКА (20-40 мин, идёт по нодам)
ansible-playbook -i inventory/bm-cluster/hosts.yaml \
  --private-key ~/.ssh/bm_k8s -u root --become \
  cluster.yml
```

ОЖИДАЕМЫЙ ВЫВОД (ХВОСТ PLAYBOOK)

```
PLAY RECAP *****
node1 : ok=512  changed=88  unreachable=0  failed=0
node2 : ok=470  changed=80  unreachable=0  failed=0
node3 : ok=395  changed=70  unreachable=0  failed=0
```

Читаем: **failed=0** и **unreachable=0** по всем нодам = кластер раскатан. Любой **failed>0** — смотри последнюю красную задачу в логе, там причина.



Забираем kubernetes на свою машину

Задача: управлять кластером со своей машины. `kubectl` не ходит по SSH — он стучится в api-server по сети (порт 6443), читая kubernetes.

LOCAL копируем admin.conf и проверяем доступ

```
mkdir -p ~/.kube

scp -i ~/.ssh/bm_k8s \
  root@node1:/etc/kubernetes/admin.conf \
  ~/.kube/bm-cluster.conf
chmod 600 ~/.kube/bm-cluster.conf

export KUBECONFIG=~/.kube/bm-cluster.conf
kubectl get nodes -o wide
kubectl get pods -A
```

ОЖИДАЕМЫЙ ВЫВОД

NAME	STATUS	ROLES	VERSION
node1	Ready	control-plane	v1.3x.x
node2	Ready	control-plane	v1.3x.x
node3	Ready	<none>	v1.3x.x

 **Грabella:** в kubernetes server = 127.0.0.1

Тогда с твоей машины будет `i/o timeout` — адрес указывает «сам на себя» на ноде.

LOCAL проверить и починить адрес api-server

```
kubectl config view --minify \
  -o jsonpath='{.clusters[0].cluster.server}'

# если там https://127.0.0.1:6443 или внутренний IP —
# замени в ~/.kube/bm-cluster.conf на публичный адрес node1
# (или адрес балансировщика), порт 6443 должен быть открыт
```



Доказываем: cri-o и Calico VXLAN

Задача: убедиться, что runtime — именно cri-o, а сеть подов — Calico в режиме VXLAN (а не на словах).

LOCAL runtime на нодах = cri-o

```
kubectl get nodes -o wide # колонка CONTAINER-RUNTIME → cri-o:// ...
```

```
ssh -i ~/.ssh/bm_k8s root@node1 'crictl ps | head'
```

```
ssh -i ~/.ssh/bm_k8s root@node3 'crictl ps | head'
```

ОЖИДАЕМЫЙ ВЫВОД

```
CONTAINER-RUNTIME
```

```
cri-o://1.3x.x # docker на нодах НЕТ — это нормально
```

CLUSTER Calico жив и в каком режиме

```
kubectl get pods -n kube-system -o wide | grep -i calico
```

```
kubectl get ippools.crd.projectcalico.org -o yaml \
  | egrep 'vxlanMode|ipipMode|cidr'
```

ОЖИДАЕМЫЙ ВЫВОД

```
vxlanMode: Always # ← оверлей VXLAN включён
```

```
ipipMode: Never
```

```
cidr: 10.233.64.0/18 # пул IP для подов
```

NODE1 интерфейс оверлея на нодe

```
ip link show | egrep 'vxlan.calico|cali|tunl0'
```

ОЖИДАЕМЫЙ ВЫВОД

```
vxlan.calico: ... # ← через него пакеты идут между нодами
```

```
caliXXXX@if...: ... # второй конец veth-пары к поду
```

СОБЕС

«Calico в режиме VXLAN: pod-пакет инкапсулируется в UDP через интерфейс `vxlan.calico`, идёт по физсети на другую ноду и там распаковывается. Проверяю режим через `ippools`, а не угадываю».



Проверяем сеть и что делать, если не взлетело

Задача: доказать, что поды на разных нодах видят друг друга, а Service резолвится по имени. Это финальная проверка живого кластера.

CLUSTER поднимаем web + service, идём из netshoot

```
kubectl create namespace net-demo
kubectl -n net-demo create deployment web --image=nginx --replicas=3
kubectl -n net-demo expose deployment web --port=80

kubectl -n net-demo get pods -o wide      # поды РАЗМАЗАНЫ по node1/2/3
kubectl -n net-demo get svc,endpoints

kubectl -n net-demo run netshoot -it --rm \
  --image=nicolaka/netshoot --restart=Never -- bash
# внутри netshoot:
curl -I web                               # 200 OK по ИМЕНИ сервиса
dig web.net-demo.svc.cluster.local +short # ClusterIP
```

ОЖИДАЕМЫЙ ВЫВОД

HTTP/1.1 200 OK # сервис нашёл поды по labels → endpoints → трафик пошёл

Если не взлетело — чем диагностировать

CLUSTER · NODE пакет диагностики

```
kubectl get nodes                # кто NotReady
kubectl describe node node1      # Conditions: Memory/DiskPressure
kubectl get pods -A | grep -v Running # что не поднялось
kubectl -n kube-system describe pod <POD>
kubectl -n kube-system logs <POD>

# на ноде (по SSH):
journalctl -u kubelet -xe | tail -40 # kubelet ругается?
journalctl -u crio -xe | tail -40    # runtime ругается?
ip route ; ss -lntp                  # маршруты и слушающие порты
```

✓ Кластер готов, если...

3 ноды **Ready** · все поды в **kube-system** Running · **curl -I web** из netshoot даёт 200 OK. Тогда по этой тетради дальше можно шупать любую тему на ЖИВОМ кластере.

Control Plane vs Worker — кто за что отвечает

Кластер = **Control Plane** (мозг, принимает решения) + **Worker Nodes** (где реально живут приложения). Ни один компонент не лезет в etcd напрямую — почти всё идёт через api-server. Разбираем по одной таблице, потом смотрим вживую на нодах.

Control Plane — мозг кластера

КОМПОНЕНТ	ЗОНА ОТВЕТСТВЕННОСТИ
api-server	Единая входная точка. Почти все запросы (kubectl, контроллеры, kubelet) идут через него. Валидирует, пишет в etcd — напрямую в etcd не лезет никто .
etcd	Память кластера. Хранит желаемое состояние (key-value, Raft consensus, нужен кворум). Единственный stateful-компонент — его бэкапят.
scheduler	Выбирает ноду для новых подов — по requests , taints/tolerations, affinity. Решает «куда», но сам не запускает.
controller-manager	Набор контроллеров. Крутит reconcile-loops : сверяет желаемое и фактическое, расхождение → действие (создать под, заменить умерший).

Worker Node — где живут приложения

КОМПОНЕНТ	ЗОНА ОТВЕТСТВЕННОСТИ
kubelet	Агент ноды. Слушает api-server, держит на ноде нужные поды, докладывает их статус наверх.
cri-o	Container runtime — реально запускает контейнеры. kubelet говорит с ним через CRI (интерфейс); cri-o или containerd — реализации.
kube-proxy	НЕ прокси-под . Заранее пишет сетевые правила (iptables/IPVS), чтобы трафик на Service попадал в нужные поды.
CNI · Calico	Сеть подов: выдаёт IP, маршруты между нодами, изоляцию. У нас Calico в режиме VXLAN.

CLUSTER смотрим ноды и роли вживую

```
kubectl get nodes -o wide # роли, версия, IP, CONTAINER-RUNTIME
```

```
# детально по одной ноде: Roles / Capacity / Allocatable / Conditions
```

```
kubectl describe node node1 | head -40
```

ОЖИДАЕМЫЙ ВЫВОД

```
Roles: control-plane
Capacity:  cpu: 2   memory: 4096Mi   # сколько физически есть
Allocatable:  cpu: 1900m  memory: 3600Mi   # сколько отдаём подам
Conditions:  MemoryPressure False   # нода здорова
              DiskPressure  False
              Ready         True
```

Читаем: **Roles: control-plane** = это мозг; **Allocatable** < **Capacity** (часть зарезервирована под систему). Именно по **Allocatable** и **requests** scheduler решает, влезет ли под.

! kube-proxy — название сбивает

Это **не** прокси-под, через который ходит трафик. kube-proxy **заранее** прописывает правила

`iptables/IPVS` на каждой ноде, а дальше ядро само заворачивает пакеты с ClusterIP на нужные поды. Сам он в проброс пакета не участвует.

kubectl — проверяем, что кластер живой

Задача: понять, что `kubectl` — это просто REST-клиент к `api-server`. Приложения пока не создаём — убеждаемся, что кластер поднялся и отвечает. Это вход в практику.

CLUSTER диагностика живого кластера

```
kubectl cluster-info           # адрес api-server и CoreDNS
kubectl get nodes              # вижу 3 ноды, все Ready
kubectl get pods -A           # системные поды во всех namespace
kubectl config current-context # в какой кластер сейчас смотрю
kubectl get componentstatuses  # health control-plane (deprecated, но наглядно)
```

ОЖИДАЕМЫЙ ВЫВОД

Kubernetes control plane is running at https://<NODE1_IP>:6443

NAME	STATUS	ROLES	VERSION
node1	Ready	control-plane	v1.3x.x
node2	Ready	control-plane	v1.3x.x
node3	Ready	<none>	v1.3x.x

NAMESPACE	NAME	READY	STATUS
kube-system	coredns-xxxx	1/1	Running
kube-system	calico-node-xxxx	1/1	Running
kube-system	kube-apiserver-node1	1/1	Running
kube-system	etcd-node1	1/1	Running

Читаем вывод: 3 ноды **Ready** + поды `kube-system` в **Running** = кластер живой и собран. Видно куб «в работе»: CoreDNS, calico-node, api-server, etcd — все служебные компоненты крутятся как обычные поды.

i componentstatuses помечен deprecated

В новых версиях вывод может быть пустым или с ошибкой — это нормально. Реальное здоровье контроллера смотрят через `kubectl get pods -n kube-system` и метрики, а не через `cs`.

СОБЕС

«`kubectl` — это REST-клиент к `api-server`, не магия. Любая команда = HTTP-запрос в `api-server`, который читает/пишет состояние в `etcd`. Живой кластер проверяю по `get nodes` (все Ready) и подам в `kube-system` (все Running)».

Умер под → вернётся ли? (self-healing)

Задача: люди говорят «Kubernetes сам поднимает поды». Точнее: он восстанавливает желаемое состояние, если оно описано через контроллер. Голый Pod (без контроллера) после удаления НЕ вернётся. Докажем оба случая руками.

CLUSTER голый Pod — удалил, НЕ вернулся

```
kubectl run solo --image=nginx # создаём ОДИНОЧНЫЙ под, без контроллера
kubectl get pods               # solo Running
```

```
kubectl delete pod solo
kubectl get pods           # пусто — никто его не пересоздаёт
```

ОЖИДАЕМЫЙ ВЫВОД

```
pod/solo created
```

NAME	READY	STATUS	AGE
solo	1/1	Running	8s

```
pod "solo" deleted
```

```
No resources found in default namespace. # желаемое состояние никто не хранил
```

CLUSTER Deployment — удалил под, ReplicaSet вернул

```
kubectl create deployment web --image=nginx --replicas=2
```

```
kubectl get pods -l app=web # два пода web-xxxx
```

```
kubectl delete pod -l app=web --field-selector=status.phase=Running \
| head -1 # грохнули ОДИН под
```

```
kubectl get pods -l app=web -w # ReplicaSet сразу поднимает замену (Ctrl-C)
```

ОЖИДАЕМЫЙ ВЫВОД

NAME	READY	STATUS	AGE	
web-aaaa	1/1	Terminating	40s	# удалённый
web-bbbb	1/1	Running	40s	
web-cccc	0/1	ContainerCreating	1s	# ← RS создал НОВЫЙ, desired=2 держится

Читаем: голый **solo** — желаемое состояние нигде не описано, удалил = исчез. **web** под Deployment: desired=2, под умер → actual=1 → ReplicaSet видит расхождение → создаёт **web-cccc**. Самовосстановление = работа контроллера, а не «магия пода».

i Цепочка восстановления — что за чем

desired (в etcd) → **actual** разошёлся → **controller** (RS) создаёт под → **scheduler** выбирает ноду → **kubelet** запускает через CRI → **CNI (Calico)** выдаёт сеть. Убери контроллер — цепочка не стартует.

Под изнутри: IP, нода, события

Задача: у пода есть IP (от Calico), но он из **внутренней** сети — изнутри кластера достучаться можно, снаружи нет. О факте пода знает только control plane. Смотрим, что про под можно вытащить и где он живёт.

CLUSTER где под, какой IP, на какой ноде

```
# подставь реальное имя пода из `kubectl get pods`
kubectl get pod web-bbbb -o wide # колонки IP и NODE

# только IP пода — точно через jsonpath
kubectl get pod web-bbbb -o jsonpath='{.status.podIP}'; echo

# полная карточка: Events / Conditions / Containers
kubectl describe pod web-bbbb
```

ОЖИДАЕМЫЙ ВЫВОД

NAME	READY	STATUS	IP	NODE	
web-bbbb	1/1	Running	10.233.92.14	node3	# IP из пула Calico

10.233.92.14 # podIP — внутренний

Conditions: Ready True

Containers: nginx Image: nginx State: Running

Events: Scheduled → Pulled → Created → Started # жизненный путь пода

pod IP — внутренний. **10.233.x.x** живёт в pod-сети Calico: другой под в кластере достучится, но **снаружи и даже по постоянному адресу — нет** (умрёт под → IP сменится). Чтобы дать стабильную точку входа — нужен **Service** (далее по тетради).

✓ Events читаются снизу вверх по времени

Scheduled (scheduler выбрал ноду) → **Pulled** (кубелет стянул образ) → **Created** → **Started**. Если под висит в **Pending** — в Events будет причина (нет ресурсов, не тянется образ, нет ноды под taint).

СОБЕС «**scheduler** выбирает ноду по **requests** (не limits) и ограничениям affinity/taints. **etcd** — единая точка правды: всё желаемое состояние лежит там, и только api-server в него пишет. podIP — внутренний, наружу выводит Service».

Доступ к кластеру: kubeconfig, context, namespace

Задача: понять, как ты вообще «заходишь» в кластер. `kubectl` — это REST-клиент к `api-server`: он читает `kubeconfig` (адрес кластера + креды + контексты) и стучится в `api-server` по сети. Здесь же щупаем **context** и **namespace**, чтобы потом не были новыми.

CLUSTER смотрим конфиг, контексты, делаем свой

```
# 1) что внутри активного конфига (адрес, юзер, namespace)
kubectl config view --minify

# 2) какие контексты вообще есть и какой активен
kubectl config get-contexts
kubectl config current-context

# 3) по фану заводим тестовый контекст: кластер + юзер + namespace
kubectl config set-context dev-ctx \
  --cluster=<cluster> --user=<user> --namespace=dev

# 4) переключаемся и смотрим поды уже в namespace dev
kubectl config use-context dev-ctx
kubectl get pods -n dev
```

ОЖИДАЕМЫЙ ВЫВОД (GET-CONTEXTS)

CURRENT	NAME	CLUSTER	AUTHINFO	NAMESPACE
*	dev-ctx	bm-cluster	admin	dev
	bm-cluster	bm-cluster	admin	

Читаем вывод: столбец **CURRENT** со звёздочкой ***** = активный контекст. Переключил `use-context` — и `kubectl` смотрит в другой кластер/namespace, не дёргая флаги каждый раз. Если `get pods -n dev` ругается `i/o timeout` — проблема не в контексте, а в адресе `api-server` (см. стр. 08 про `server: 127.0.0.1`).

і Два понятия сразу

context — запомненная связка «кластер + юзер + namespace по умолчанию»; переключаешь контекст — меняешь, куда смотришь. **namespace** — логическая «папка» внутри ОДНОГО кластера (`dev` / `prod` / `staging`): своя изоляция объектов, прав и квот. `admin.conf` = root всего кластера, в прод его не раздают.

СОБЕС

« `kubectl` — REST-клиент к `api-server`, `kubeconfig` = адрес + ключи + контексты. Context — это связка кластер+юзер+namespace, namespace — логическое разделение внутри одного кластера. На работе: дали `kubeconfig` → положил → `use-context` → `get nodes` ».

RBAC: выдаём ограниченный доступ

Задача: со стороны АДМИНА выдать кому-то (девопсу) **ограниченный** доступ — только читать поды в одном namespace, а не root на весь кластер. Делается через **ServiceAccount + Role + RoleBinding**, а проверяется БЕЗ риска через `auth can-i ... --as=...`.

CLUSTER SA + Role + RoleBinding и проверка прав

```
# 1) рабочая область и «юзер для процессов» внутри кластера
kubectl create namespace dev
kubectl -n dev create serviceaccount dev-reader

# 2) Role = ЧТО можно (только читать поды)
kubectl -n dev create role pod-reader \
  --verb=get,list,watch --resource=pods

# 3) RoleBinding = КОМУ привязываем эту роль
kubectl -n dev create rolebinding dev-reader-b \
  --role=pod-reader --serviceaccount=dev:dev-reader

# 4) проверяем права за этого SA, ничего не ломая
kubectl auth can-i list pods \
  --as=system:serviceaccount:dev:dev-reader -n dev
kubectl auth can-i delete pods \
  --as=system:serviceaccount:dev:dev-reader -n dev
```

ОЖИДАЕМЫЙ ВЫВОД

```
yes      # list pods — роль это разрешает
no       # delete pods — в Role такого verb нет
```

Читаем вывод: `yes` / `no` — это ответ авторизации (RBAC) на вопрос «можно ли». `--as=` подставляет личность SA, не выдавая тебе его токен — удобно проверять права заранее. `delete → no`, потому что в Role перечислены только `get,list,watch`.

i Порядок внутри api-server

`authn → authz → admission`. `authn` (кто ты — TLS-сертификат или токен) → `authz` (что можно — RBAC) → `admission` (может изменить/запретить запрос). **Role/RoleBinding** действуют в рамках namespace; **ClusterRole/ClusterRoleBinding** — на весь кластер. **ServiceAccount** — это «юзер» для подов и процессов.

СОБЕС

«Фишка прода — ограниченные ServiceAccount с минимальными правами, а НЕ раздавать всем `admin.conf` (это root всего кластера). Role говорит ЧТО можно, RoleBinding — КОМУ. Проверяю права без риска через `kubectl auth can-i --as=...`».

Control plane умер — что живёт, что ломается

Любимый вопрос на собеседовании. Люди думают «упал control plane — упал кластер». На деле **data plane живёт по инерции**, но управлять кластером уже нельзя. Разберём по полочкам, что именно отваливается.

✓ ЖИВЁТ (data plane)

- запущенные поды **работают**, трафик идёт
- `kube-proxy` -правила уже лежат на нодах
- `kubelet` автономен — держит свои поды сам

🔧 ЛОМАЕТСЯ (управление)

- нет `apply` новых манифестов, нет `scale`
- `scheduler` не ставит новые поды
- изменения не пишутся в `etcd`
- self-healing мёртв**: `controller-manager` лежит вместе с control plane

Кластер живёт по инерции, но любой **НОВЫЙ** сбой уже не лечится: упадёт под прямо сейчас — не вернётся, потому что некому сравнить desired и actual. Поэтому control plane делают отказоустойчивым — **3 мастера** (чтобы потеря одного не уронила управление).

і Почему именно self-healing отваливается

Самовосстановление = работа `controller-manager`: он видит расхождение desired↔actual и создаёт новый под. Он часть control plane → control plane лёг → сравнивать некому → новый под не появится, хотя старые продолжают крутиться.

СОБЕС

«Поды работают, управлять нельзя. Data plane (kubelet + kube-proxy) автономен и держит нагрузку по инерции; но `apply` / `scale` / scheduling / self-healing мертвы, запись в etcd не идёт. Любой новый сбой не лечится — поэтому control plane делают на 3 мастера».

Задача: etcd на 5 нод, 3 упали

Классическая задача на собес. Чтобы её решить, надо понимать **кворум** и **Raft**. Сначала теория одним абзацем, потом считаем — и проверяем руками на ноде.

Считаем кворум через Raft

Кворум — БОЛЬШИНСТВО нод, которое должно согласиться, чтобы запись считалась принятой: для N нод = $N/2 + 1$ (5 → 3, 3 → 2).

Raft (консенсус etcd) в 3 шага: ① среди нод выбирают одного **лидера** (leader election); ② все записи идут через лидера, он реплицирует их на followers; ③ запись **коммитится**, только когда её подтвердило большинство = кворум.

Задача: 5 нод → кворум 3. Упали 3 → осталось $2 < 3$ → кворум потерян → лидера не выбрать и записи не закомитить → по сути **read-only**. При этом запущенные поды работают (kubelet автономен), вносить изменения нельзя.

NODE1 проверяем здоровье и кворум etcd

```
# подставь свои серты/endpoints (см. блок про etcd на стр. 11)
ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  endpoint health

etcdctl member list          # список членов кластера etcd
etcdctl endpoint status -w table # кто LEADER, размер БД

ОЖИДАЕМЫЙ ВЫВОД (КВОРУМ ПОТЕРЯН)
https://127.0.0.1:2379 is unhealthy: ... context deadline exceeded
# в endpoint status таблице: LEADER нет → записи не проходят
```

Читаем вывод: **unhealthy** + отсутствие **LEADER** в таблице = большинство недоступно, кластер ушёл в read-only. Если живых нод $\geq$ кворума — **endpoint health** вернёт **is healthy** и в **status** будет виден лидер.

i Чинить и почему нечётное число

Поднять упавшие ноды обратно — или восстановить из бэкапа: **etcdctl snapshot restore**. Число нод берут **нечётным**: 5 и 6 терпят по 2 отказа — шестая надёжности не добавляет, только повышает риск split-brain.

СОБЕС

«Кворум → нет записей → нагрузка живёт. Для N нод кворум = $N/2+1$; 5 нод теряют кворум при падении 3 (осталось $2 < 3$) → etcd read-only, лидера не выбрать. Поды работают, изменения не сохраняются. Чиню: вернуть ноды или **snapshot restore**. Число нод — нечётное».

Заглядываем в etcd напрямую

Задача: доказать, что etcd — это память кластера, где лежит ВСЁ желаемое состояние. У нас (kubespary) etcd — это systemd-сервис на node1, поэтому `etcdctl` запускаем прямо на ноде, не через kubectl.

Сертификаты не угадываем — берём из процесса

ФЛАГ У ETCD (В PS)	→ ФЛАГ ДЛЯ ETCDCTL	ЧТО ЭТО
<code>--trusted-ca-file</code>	<code>--cacert</code>	корневой CA
<code>--cert-file</code>	<code>--cert</code>	клиентский сертификат
<code>--key-file</code>	<code>--key</code>	приватный ключ
<code>--advertise-client-urls</code>	<code>--endpoints</code>	адрес etcd

NODE1 находим сертификаты и читаем реестр

```
# 1) подсмотреть пути сертов прямо в запущенном процессе etcd
ps -efww | grep '[e]tcd'
ls /etc/ssl/etcd/ssl/           # точные имена node-node1.pem и т.п.
```

```
# 2) сложить 4 флага и спросить, что лежит в кластере
ETCDCTL_API=3 etcdctl \
  --cacert=/etc/ssl/etcd/ssl/ca.pem \
  --cert=/etc/ssl/etcd/ssl/node-node1.pem \
  --key=/etc/ssl/etcd/ssl/node-node1-key.pem \
  --endpoints=https://127.0.0.1:2379 \
  get /registry --prefix --keys-only | head
```

ОЖИДАЕМЫЙ ВЫВОД

```
/registry/pods/net-demo/web-xxxx
/registry/deployments/net-demo/web
/registry/services/...           # ВСЁ состояние кластера лежит здесь
```

Читаем: ключи `/registry/pods/...`, `/registry/deployments/...` — это и есть желаемое состояние. Пишет сюда ТОЛЬКО api-server, напрямую в etcd не лезет никто.



Собес-ловушка: Secret в etcd зашифрован?

По умолчанию НЕТ — лежит base64. Тем же `etcdctl get /registry/secrets/...` пароль виден открытым. Защита: **encryption-at-rest** на api-server + RBAC + Vault.

Сначала координаты, потом Pod

Новичок применяет YAML и не понимает: куда он его применил и почему `get pods` пустой. Ответ — координаты. Pod не висит в воздухе: он живёт в конкретном кластере, контексте и namespace. Сначала разбираемся «где мы», потом создаём ресурс.

i Сначала координаты — три уровня

kubeconfig — файл доступа (кластеры / пользователи / контексты). **context** — выбранная связка «кластер + пользователь + namespace по умолчанию». **namespace** — логическая «папка» ресурсов внутри кластера.

Формула: `kubectl apply -f` → current context → cluster + user + namespace → ресурс.

Разведи два namespace

Linux namespace (изоляция процесса, ядро) ≠ **Kubernetes Namespace** (логическое пространство ресурсов в кластере). Слово одно — сущности разные.

Pod — минимальная единица запуска. Внутри один или несколько контейнеров: общий IP, видят друг друга по **localhost**, делят **volume**, стартуют и умирают вместе. Тонкость на собес: в поде всегда есть скрытый **pause-контейнер** — поднимается первым и держит сетевой namespace. Напрямую подами обычно не управляют — это делают контроллеры.

CLUSTER проверяем координаты → создаём Pod → читаем

```
# 1) где я сейчас? какой контекст активен
kubectl config current-context

# 2) какие namespace есть в кластере (это «папки»)
kubectl get namespaces

# 3) применяем под В КОНКРЕТНЫЙ namespace (-n dev), не в воздух
kubectl apply -f pod.yaml -n dev

# 4) смотрим под + на какой ноде и с каким Pod IP он живёт
kubectl get pods -n dev -o wide

# 5) доказываем, какие контейнеры реально внутри пода
kubectl get pod app -n dev \
  -o jsonpath='{.spec.containers[*].name}'
```

ОЖИДАЕМЫЙ ВЫВОД

NAME	READY	STATUS	NODE	IP
app	1/1	Running	node3	10.233.x.x
app				# имя контейнера из шага 5

Читаем вывод: `get pods` пустой почти всегда = смотришь не в тот namespace (забыл `-n dev`) или не в тот контекст. `-o wide` показывает ноду и Pod IP — доказательство, что под реально приземлился на узел.

СОБЕС

«Pod не существует сам по себе: создаётся в кластере → контексте → namespace. Внутри — контейнеры с общим IP, localhost и volume; первым стартует pause-контейнер и держит сетевой namespace. Напрямую подами не управляю — это работа контроллеров».

ReplicaSet: self-healing своими руками

Задача: доказать руками, что такое desired state. **ReplicaSet** — первый контроллер с нормальной целью: «держи N одинаковых подов». Реплик меньше — создаёт новые, больше — лишние убирает. Прибиваем под и смотрим, как RS сам поднимает замену.

Свои поды RS находит через **labels** и **selector** (не угадывает по имени), создаёт из **template**. Руками RS почти не делают — его создаёт **Deployment** (тот лучше управляет обновлениями и откатами). Здесь создаём напрямую, только чтобы увидеть механику самовосстановления.

CLUSTER RS на 2 реплики → убиваем под → ловим замену

```
# 1) применяем ReplicaSet (replicas: 2, selector app=web)
kubectl apply -f rs.yaml
```

```
# 2) видим сам RS и его поды разом, фильтр по метке
kubectl get rs,pods -l app=web
```

```
# 3) ЛОМАЕМ: прибиваем ОДИН под (подставь реальное имя)
kubectl delete pod <one>
```

```
# 4) смотрим в реальном времени — RS создаёт новый под
kubectl get pods -w          # Ctrl+C чтобы выйти
```

```
# 5) по какому селектору RS вообще ищет свои поды
kubectl get rs web \
  -o jsonpath='{.spec.selector}'
```

ОЖИДАЕМЫЙ ВЫВОД

NAME	DESIRED	CURRENT	READY	AGE
web	2	2	2	1m

```
# после delete: один под Terminating, тут же новый ContainerCreating → Running
{"matchLabels":{"app":"web"}}
```

Читаем вывод: **DESIRED 2 • CURRENT 2 • READY 2** — фактическое совпало с желаемым. Убил под — **CURRENT** на миг стал 1, контроллер увидел расхождение и поднял новый: фактическое снова подтянулось к 2. Это и есть desired state живьём.

і Как RS находит «свои» поды

RS не помнит поды по имени — он сравнивает **labels** подов со своим **selector**. Поэтому новый под с теми же метками автоматически становится «его». Руками RS почти никогда не создают: на проде делают **Deployment**, а он уже сам создаёт ReplicaSet под капотом.

СОБЕС

«ReplicaSet держит N подов по селектору: видит, что живых меньше desired, — создаёт новые из template; больше — удаляет лишние. Поды находит по labels, не по имени. Напрямую его не пишу — создаю Deployment, RS он сделает сам».

Pod = приложение + его помощники

Pod нужен НЕ потому что куб не умеет в контейнеры. Pod нужен, когда логический сервис — это **приложение + технические помощники рядом**. Они в одном поде → куб ставит их на одну ноду, запускает вместе, даёт общий volume и общий сетевой namespace. Поэтому помощник видит файлы приложения и ходит к нему по localhost.

Четыре вида контейнеров в Pod

ВИД	КОГДА ЖИВЁТ	ЗАЧЕМ
App	всё время, обязателен	основное приложение (backend / API / worker), обычно один
Sidecar	ВМЕСТЕ с app, рядом	помощник: логи, метрики, проху; общий volume + localhost
Init	ДО app → отработал → умер	дождаться базы, накатить миграции, подготовить файлы
Ephemeral	подсажен в РАБОТАЮЩИЙ под	debug «на живую» (<code>kubectl debug</code>), особенно для distroless без shell

CLUSTER общий volume + sidecar + ephemeral debug

```
# 1) app пишет строки в файл на emptyDir,
#   sidecar (busybox) делает tail -f того же файла
kubectl apply -f pod-with-sidecar.yaml

# 2) читаем логи КОНКРЕТНОГО контейнера в поде (-c sidecar)
kubectl logs app -c sidecar

# 3) подсаживаем временный debug-контейнер в работающий под
kubectl debug -it app \
  --image=netshoot --target=app
```

ОЖИДАЕМЫЙ ВЫВОД

```
line 1 from app      # sidecar видит строки, что пишет app
line 2 from app      # → общий emptyDir работает
# debug: попадаем в shell netshoot в сетевом namespace пода app
```

Читаем вывод: sidecar показывает строки, которые писал **другой** контейнер, — доказательство общего volume (emptyDir). `--target=app` сажает ephemeral-контейнер в namespace процессов app, поэтому из netshoot видно его сеть и процессы.

i init vs sidecar — частый вопрос

Init отработывает и **умирает ДО** старта app (последовательно, по очереди). **Sidecar** живёт **вместе** с app всё время. В k8s 1.28+ появился «native sidecar» — это restartable init-контейнер: объявляется как init, но не завершается, а работает рядом с app.

СОБЕС

«Pod = приложение + ближайшие помощники на одной ноде с общим volume и localhost. Backend и Postgres — РАЗНЫЕ поды; backend и sidecar для логов — ОДИН. Init умирает до app, sidecar живёт с ним, ephemeral подсаживаю в работающий под для отладки».

Сеть пода: eth0 ↔ veth ↔ cali

Задача: доказать руками, что `eth0` внутри пода — это не физкарта, а **один конец veth-пары**. Второй конец воткнут в ноду и называется `caliXXXX`. Calico = routed networking (маршруты, а не docker0-bridge). Заходим в под, потом находим парный интерфейс на ноде.

У Pod свой сетевой namespace: свой `eth0`, свой IP (/32), свои маршруты. `eth0` одним концом смотрит в под, другим — на ноду. По индексу `@if<N>` из пода мы находим тот самый второй конец на узле — это и есть «вау-момент».

POD смотрим интерфейс и маршруты ИЗНУТРИ пода

```
# заходим в под net-a (netshoot уже с утилитами)
kubectl exec -it net-a -- sh

# внутри пода:
ip -br addr          # eth0 + Pod IP /32, коротким списком
ip route             # default via 169.254.1.1 — шлюз Calico
ip -o link show eth0 # eth0@if<N> — N это индекс пары на ноде
```

ОЖИДАЕМЫЙ ВЫВОД (ВНУТРИ ПОДА)

```
eth0    UP    10.233.x.x/32
default via 169.254.1.1 dev eth0
N: eth0@if123: ...    # ← запоминаем число N (тут 123)
```

NODE находим ВТОРОЙ конец veth на ноде по N

```
# на ноде, где живёт под, ищем интерфейс с индексом N
ip -o link show | grep '^<N>:'
# подставь своё число вместо <N>, напр. ^123:
```

ОЖИДАЕМЫЙ ВЫВОД (НА НОДЕ)

```
123: caliXXXXXXXXXX@if4: <BROADCAST,...> ...
# ↑ это и есть второй конец veth-пары к поду net-a
```

Вау-момент: `eth0@if123` в поде и `123: caliXXXX` на ноде — это два конца ОДНОГО виртуального кабеля. Пакет из пода уходит в `eth0` и мгновенно выныривает на ноде в `caliXXXX`. Никакой магии — просто veth-пара.

i CNI = только pod-to-pod

Calico (CNI) отвечает **только** за связность между подами: Pod IP, veth-пара, маршруты в поде и на ноде.

Service / ClusterIP — это следующий слой (kube-proxy + iptables/IPVS), CNI его не делает. Не путай «под видит под» и «сервис резолвится по имени».

Между нодами: пакет уходит в VXLAN

Задача: доказать руками разницу «свой узел / чужой узел». Пакет к поду на **другой** ноде заворачивается в `vxlan.calico` (инкапсуляция в UDP), летит по физсети и распаковывается на той ноде. Пакет к поду на **той же** ноде уходит напрямую через `caliXXXX` и физсеть не трогает. Сравниваем маршрут той же командой.

POD • NODE какой интерфейс выберет Linux для каждого Pod IP

```
# 1) маршрут до пода на ДРУГОЙ ноде → оверлей VXLAN
ip route get <pod-на-другой-ноде>
#   ожидаем: dev vxlan.calico — пакет инкапсулируется

# 2) маршрут до пода на ТОЙ ЖЕ ноде → напрямую через cali
ip route get <pod-на-той-же-ноде>
#   ожидаем: dev caliXXXX — в физсеть НЕ уходит

# 3) доказываем режим оверлея, а не угадываем
kubectl get ip pools.crd.projectcalico.org -o yaml \
  | egrep 'vxlanMode|ipipMode'
```

ОЖИДАЕМЫЙ ВЫВОД

```
10.233.a.b dev vxlan.calico src ... # другой узел → VXLAN
10.233.c.d dev caliXXXXXXXXXX src ... # тот же узел → напрямую
vxlanMode: Always # ← оверлей включён
ipipMode: Never
```

Читаем вывод: одна и та же команда `ip route get` даёт **разный** интерфейс — Linux сам выбирает по таблице маршрутов. `dev vxlan.calico` = чужая нода, нужен оверлей; `dev caliXXXX` = сосед на этом же узле, физсеть не нужна. (Вариант `dev tunl0` = режим IPIP; `via <node-ip>` = native BGP.)

✓ Формула пути пакета — запомни целиком

eth0 (в поде) → veth → cali (на ноде) → vxlan.calico → другой Pod. Локальные адреса уходят на `caliXXXX` и остаются внутри ноды; удалённые — на `vxlan.calico`, инкапсулируются в UDP и едут по физсети на другую ноду, где распаковываются.

СОБЕС

«Calico в режиме VXLAN: пакет к поду на другой ноде заворачивается в UDP через `vxlan.calico`, идёт по физсети и там распаковывается; к поду-соседу на той же ноде — напрямую через `cali`, без оверлея. Режим проверяю через `ip pools` (`vxlanMode: Always`), а не угадываю».

Deployment · StatefulSet · DaemonSet — кто для чего

Под напрямую в проде никто не создаёт — его оборачивают в контроллер. Три героя, три разных задачи. Запоминаешь не определение, а **зону применения**: спросят именно «когда что брать».

КОНТРОЛЛЕР	ДЛЯ ЧЕГО	ИМЕНА ПОДОВ	ДИСК	СУТЬ В ОДНОЙ ФРАЗЕ
Deployment	stateless-приложения	случайные (web-7d9f-x2k)	общий / без диска	обёртка над ReplicaSet: rolling update + rollback, история ревизий
StatefulSet	stateful (БД, Kafka, Redis)	стабильные (web-0, web-1)	свой PVC на каждый под	порядок создания 0→N, диск не теряется при пересоздании
DemonSet	агенты на нодах	по поду на ноду	обычно hostPath	лог-агенты, мониторинг, CNI — по одному на КАЖДУЮ подходящую ноду

Задача: развернуть stateless-приложение через Deployment, увидеть связку

Deployment → ReplicaSet → Pod, отмасштабировать его и посмотреть, какие StatefulSet/DaemonSet уже живут в кластере.

CLUSTER создаём Deployment и смотрим связку

```
# 1) одной командой: Deployment web, образ nginx, 3 реплики
kubectl create deployment web --image=nginx --replicas=3

# 2) видим всю цепочку разом: Deployment → ReplicaSet → Pods
kubectl get deploy,rs,pods

# 3) масштабируем 3 → 5 (контроллер сам доводит до желаемого)
kubectl scale deploy/web --replicas=5
kubectl get pods -l app=web

# 4) кто из StatefulSet/DaemonSet уже стоит во всём кластере
kubectl get statefulset,daemonset -A
```

ОЖИДАЕМЫЙ ВЫВОД

```
deployment.apps/web      5/5      5      5
replicaset.apps/web-7d9f  5        5      5      # 1 RS = 1 версия образа
pod/web-7d9f-xxxxx       1/1      Running  # *5, имена случайные
```

```
NAMESPACE   NAME                                DESIRED  CURRENT  READY
kube-system  daemonset.apps/calico-node         3        3        3      # по поду на ноду
kube-system  daemonset.apps/kube-proxy          3        3        3
```

Читаем вывод: один **ReplicaSet** = одна версия образа; поды носят его хеш в имени. **scale** меняешь желаемое число — контроллер сам создаёт/удаляет поды. DaemonSet **calico-node** и **kube-proxy** стоят на всех 3 нодах: это и есть «по поду на ноду».

i StatefulSet — стабильные имена + свой диск

У Deployment поды одноразовые и взаимозаменяемые. У StatefulSet под `web-0` при пересоздании остаётся `web-0` и получает обратно СВОЙ `PVC` (через `volumeClaimTemplates`). Поэтому туда сажают БД: диск переживает пересоздание пода.

СОБЕС

«Deployment — для stateless: rolling update и rollback через ReplicaSet. Говорю **stateful**, а не «для БД» — StatefulSet даёт стабильные имена `web-0/1/2` и персональный PVC каждому поду. DaemonSet — по поду на каждую ноду, для агентов».

Rolling update, откат и вывод ноды на обслуживание

Задача: обновить образ Deployment без даунтайма, посмотреть, как новый ReplicaSet растёт, а старый уходит в 0, и при поломке откатиться одной командой. Это спрашивают почти всегда.

CLUSTER обновляем образ → следим → откатываем

```
# 1) меняем образ — стартует rolling update
kubectl set image deploy/web nginx=nginx:1.27

# 2) ждём, пока выкатка дойдёт до конца
kubectl rollout status deploy/web

# 3) два ReplicaSet: новый растёт, старый сдувается в 0
kubectl get rs

# 4) что-то сломалось → откат на предыдущую ревизию
kubectl rollout undo deploy/web

# 5) история ревизий (что и когда катили)
kubectl rollout history deploy/web
```

ОЖИДАЕМЫЙ ВЫВОД

deployment "web" successfully rolled out

NAME	DESIRED	CURRENT	READY	
web-7d9f	0	0	0	# старая версия — сдулась
web-86b4	5	5	5	# новая версия — выросла

Читаем вывод: два ReplicaSet — это норма, не баг. Старый держится в 0 реплик ради быстрого **undo**. Шаг выкатки задают **maxUnavailable** (сколько можно потерять в моменте) и **maxSurge** (сколько поднять сверх replicas).

Задача: вывести ноду на обслуживание так, чтобы приложение не упало ниже минимума живых подов. Для этого — PDB, затем cordon/drain.

CLUSTER PDB + вывод ноды (cordon → drain)

```
# 1) PDB: держать минимум 2 живых пода web при выгоне
kubectl create poddisruptionbudget web-pdb \
  --selector=app=web --min-available=2

# 2) сколько подов МОЖНО выключить прямо сейчас
kubectl get pdb          # ALLOWED DISRUPTIONS = 3 (5 живых - min 2)

# 3) cordon: на node3 НЕ слать новые поды (старые работают)
kubectl cordon node3

# 4) drain: мягко выгнать поды с node3, daemonset не трогаем
kubectl drain node3 --ignore-daemonsets
```

ОЖИДАЕМЫЙ ВЫВОД

```
poddisruptionbudget.policy/web-pdb created
```

NAME	MIN AVAILABLE	ALLOWED DISRUPTIONS
web-pdb	2	3

```
node/node3 cordoned
evicting pod default/web-86b4-xxxxx # drain ждёт, чтобы не нарушить PDB
```

⚠ Две разные ручки — не путай

`maxUnavailable/maxSurge` рулят САМИМ rolling update (плавность выкатки). **PDB** — отдельный объект: ограничивает, сколько подов выключено ОДНОВРЕМЕННО при добровольных disruptions (drain, обновление, autoscaler). `drain` ждёт, если выгон нарушит PDB. Но PDB **≠ персистентность**: «файл переживает под» — это PV/PVC, а PDB про доступность подов и от краша ноды/OOM не спасает.

СОБЕС

«Как обновить кластер без даунтайма для приложения?» → **PDB** (минимум живых подов) + `cordon/drain` ноды по очереди. На проде откат делаю через Helm/CI, а не `kubectl rollout undo` руками».

Service и Ingress — стабильная точка входа

Поды эфемерны и меняют IP при пересоздании. **Service** даёт стабильную точку входа и раскидывает трафик по живым подам, находя их **по labels**, а не по имени. Пять типов — пять способов «как тебя найдут».

ТИП SERVICE	ОТКУДА ДОСТУПЕН	СУТЬ В ОДНОЙ ФРАЗЕ
ClusterIP	только внутри кластера	тип по умолчанию; доступ по DNS-имени между подами
NodePort	снаружи, порт на каждой ноде	один и тот же порт на КАЖДОЙ ноде; так бьёт внешний HAProxy
LoadBalancer	снаружи, внешний IP	просит IP у облака/MetalLB; у нас висит <pending> — MetalLB нет
Headless	внутри, без ClusterIP	DNS отдаёт IP конкретных подов (для StatefulSet)
ExternalName	наружу, как CNAME	не ведёт в поды — DNS-алиас на внешний адрес (гибрид облако↔железо)

Задача: повесить ClusterIP-сервис на Deployment **web**, увидеть, что он нашёл поды (EndpointSlice не пуст), и достучаться до него по ИМЕНИ изнутри кластера.

CLUSTER expose → endpoints → curl по имени

```
# 1) Service для Deployment web (ClusterIP, порт 80)
kubectl expose deploy/web --port=80

# 2) Service + актуальный список живых backend'ов
kubectl get svc,endpointslice

# 3) узнаём ClusterIP сервиса напрямую
kubectl get svc web -o jsonpath='{.spec.clusterIP}'

# 4) одноразовый под netshoot → стучимся по ИМЕНИ web
kubectl run t --rm -it --image=netshoot -- curl -I web
```

ОЖИДАЕМЫЙ ВЫВОД

```
service/web ClusterIP 10.233.21.7 80/TCP
endpointslice.discovery.k8s.io/web-abcde IPv4 80 10.233.x.x, ...
10.233.21.7
HTTP/1.1 200 OK # сервис нашёл поды по labels → endpoints → трафик пошёл
```

Читаем вывод: **curl -I web** по короткому имени даёт 200 — значит цепочка

labels → EndpointSlice → kube-proxy собралась. Если EndpointSlice пуст — Service не нашёл поды (чаще всего разъехался selector).

i Ingress = правило + контроллер; и про source IP

Ingress — это ПРАВИЛО маршрутизации HTTP (host/path), а **Ingress-контроллер** (nginx/Traefik) — тот, кто его исполняет. Сам Ingress трафик не принимает. Цепочка: **Ingress → Service → поды**. У NodePort/LoadBalancer спрашивают **externalTrafficPolicy**: **Cluster** (по умолч.) балансирует на под на любой ноде — возможен лишний хоп, source IP клиента теряется (SNAT); **Local** — только на под на той же ноде, source IP СОХРАНЯЕТСЯ, но нужен под на каждой ноде. Все типы создаём манифестами (`kubectl apply -f`), не `expose`.

СОБЕС

«Service находит поды по **labels** и держит стабильный ClusterIP/DNS-имя. Ingress сам по себе ничего не делает — это правило, нужен Ingress-контроллер. Нужен реальный IP клиента → **externalTrafficPolicy: Local**, ценой пода на каждой ноде».

ConfigMap и Secret — настройки отдельно от образа

Задача: вынести настройки и секреты ИЗ образа, чтобы менять их без пересборки.

Несекретное → **ConfigMap**, чувствительное (пароли, токены, ключи) → **Secret**. И тут же доказать, что Secret — это НЕ шифрование.

CLUSTER создаём конфиг и секрет, читаем пароль

```
# 1) ConfigMap: несекретная настройка приложения
kubectl create configmap app-cfg --from-literal=MODE=prod

# 2) Secret: чувствительное значение (пароль к БД)
kubectl create secret generic db --from-literal=PASSWORD=s3cr3t

# 3) достаём пароль обратно — base64 раскодируется одной командой
kubectl get secret db -o jsonpath='{.data.PASSWORD}' | base64 -d
```

ОЖИДАЕМЫЙ ВЫВОД

```
configmap/app-cfg created
secret/db created
s3cr3t      # ← пароль ОТКРЫТЫМ текстом: base64 защитой не был
```

Читаем вывод: в `.data.PASSWORD` лежит `czNjcjN0` — это base64, а `base64 -d` возвращает исходный `s3cr3t`. Любой с доступом к Secret раскодирует пароль в одну строку.

🕒 Собес-ловушка: Secret = зашифровано?

НЕТ. base64 — это кодировка, а не шифрование: декодируется без ключа. Реально защищают: **encryption-at-rest** на api-server (тогда в etcd лежит шифротекст) + **RBAC** (кто вообще видит Secret) + в проде **Vault** / External Secrets. Плюс Secret монтируется в память (tmpfs), не на диск ноды.

і env vs volume — важный нюанс

И ConfigMap, и Secret подключают двумя способами: как **env-переменные** или как **файлы через volume**. Разница: уже запущенный под НЕ подхватит изменение через **env** — нужен рестарт Pod/Deployment. Файлы из **volume** куб обновит на лету, но приложение должно само их перечитать.

СОБЕС

«ConfigMap — несекретные настройки, Secret — чувствительное, и то и другое вынесено из образа. **base64 ≠ шифрование**: показываю декод в одну команду. Шифрую через encryption-at-rest + RBAC + Vault. env-переменные требуют рестарта пода, volume-файлы — нет».

Volume · PV · PVC — данные переживают под

Задача: доказать главный смысл PV/PVC — под эфемерен, а данные нет. Записываем файл в смонтированный том, УБИВАЕМ под, пересоздаём — и файл на месте. Без PVC он умер бы вместе с подом.

CLUSTER пишем файл → убиваем под → файл на месте

```
# 1) заявка на диск (PVC) и под, который её монтирует
kubectl apply -f pvc.yaml

# 2) PVC получил реальный диск — статус Bound, виден PV
kubectl get pvc,pv

# 3) пишем файл в смонтированный том /data
kubectl exec demo -- sh -c 'echo hi > /data/t.txt'

# 4) УБИВАЕМ под целиком
kubectl delete pod demo

# 5) пересоздаём под — PVC/диск тот же самый
kubectl apply -f pod.yaml

# 6) читаем файл в новом поде — он НА МЕСТЕ
kubectl exec demo -- cat /data/t.txt
```

ОЖИДАЕМЫЙ ВЫВОД

```
persistentvolumeclaim/demo-pvc   Bound    pvc-abc123   1Gi   RWO
persistentvolume/pvc-abc123       1Gi     RWO         Bound
```

```
pod "demo" deleted
```

```
pod/demo created
```

```
hi      # ← файл пережил удаление пода: данные на PVC, не в поде
```

Читаем вывод: **PVC = Bound** — заявка нашла реальный диск (PV). После **delete pod** + пересоздания **cat** отдаёт **hi** — значит данные жили на PVC, а не внутри пода. Это и есть «персистентность».

⚠ On-prem без StorageClass → PVC висит Pending

На голом kubernetes/kubeadm HET дефолтного StorageClass и provisioner'a, который создаёт PV → PVC останется в **Pending** навсегда, под не стартует. В облаке (GKE/EKS) есть CSI-драйвер + default StorageClass, диск создаётся сам. On-prem ставишь storage сам: **local-path** (dev, папки на ноде), **NFS** (сетевой, RWX), **Ceph/Rook** или **Longhorn** (распределённое для прода).

i Цепочка и режимы доступа

Pod → PVC (заявка «дайте N ГБ») → **StorageClass** (динамически создаёт) → **PV** (реальный диск). Под монтирует именно **PVC**, не PV напрямую. Режимы: **RWO** — диск у одной ноды (обычный блок); **RWX** — пишут много нод сразу (нужен NFS/CephFS). StatefulSet выдаёт каждому поду свой PVC через **volumeClaimTemplates**.

СОБЕС

«PV/PVC = данные переживают под: показываю записью файла → удалением пода → файл на месте. Цепочка **Pod→PVC→StorageClass→PV**, монтируется PVC. On-prem без provisioner'a PVC висит в Pending — это **не** PDB (PDB про доступность при drain)».

Связка целиком: как всё держится на labels

Задача: увидеть на ЖИВОМ кластере, как объекты сцеплены через метки. `app=web` — это нитка, на которой висит вся цепочка: Service находит поды по labels, EndpointSlice хранит живые, kube-proxy и Ingress доводят трафик.

CLUSTER тянем за один label — поднимается вся связка

```
# 1) всё, что связано меткой app=web, разом — с самими лейблами
kubectl get deploy,rs,pods,svc,endpointslice \
  -l app=web --show-labels
```

```
# 2) какой EndpointSlice обслуживает именно Service web
kubectl get endpointslice \
  -l kubernetes.io/service-name=web
```

ОЖИДАЕМЫЙ ВЫВОД

```
deployment.apps/web      5/5      ...    app=web
replicaset.apps/web-86b4  5        ...    app=web,pod-template-hash=86b4
pod/web-86b4-xxxxx       1/1      Running app=web, ...
service/web              ClusterIP 10.233.21.7 ...
endpointslice/web-abcde  IPv4     80     10.233.x.x, ... # ← живые PodIP здесь
```

Читаем связку: `Deployment` → поды (по pod-template) → `Service` цепляет их по `app=web` → `EndpointSlice` хранит живые PodIP:port → `kube-proxy` пишет сетевые правила → `Ingress` — контроллер роутит HTTP снаружи. Один label держит всё.

🔧 Топ-грабля: разъехался selector → endpoints пустеют

Поменял labels на подах или selector на Service — Service перестаёт находить поды, `EndpointSlice` становится пустым, и трафик молча перестаёт идти (под жив, сервис «есть», а 503/timeout). Первое, что смотришь при «сервис не отвечает»: `kubectl get endpointslice -l kubernetes.io/service-name=web` — если пусто, ищи расхождение labels ↔ selector. Идеальный кейс для разбора через ИИ.

СОБЕС

«Всё в кубе связано **labels**, не именами: Deployment делает поды → Service находит их по селектору → EndpointSlice держит живые → kube-proxy раздаёт правила → Ingress роутит. Сломал selector — endpoints пустеют, трафик встал. Это первое, что проверяю при «сервис не отвечает».

kubectl apply — что реально происходит

Когда ты пишешь `kubectl apply`, ты не идёшь на сервер и не запускаешь контейнер. Ты отправляешь в кластер описание желаемого состояния — а дальше внутри прокручивается цепочка из пяти звеньев. Разберём её по атомам, а потом проследим вживую командами.

Цепочка apply — пять звеньев

- 1 **① kubectl → api-server.** Уходит desired state («хочу Deployment, 2 реплики, такой образ»). api-server прогоняет: **authentication** (кто ты) → **authorization** (RBAC: что тебе можно) → **admission** (вебхуки могут изменить или отклонить) → **validate** схемы → пишет объект в `etcd`. Контейнера ещё НЕТ — есть запись «вот что должно быть».
- 2 **② controller-manager.** Контроллеры **watch'at** api-server. Deployment-controller видит новый Deployment → создаёт **ReplicaSet**. ReplicaSet-controller видит «надо N, есть 0» → создаёт **Pod objects**. Но это пока объекты: `nodeName` пусто, контейнеров не запущено.
- 3 **③ scheduler.** Ищет поды без `nodeName`. Две фазы: **filter** (отсеять ноды, куда нельзя — хватает ли requests, taints, nodeSelector) → **score** (ранжировать, выбрать лучшую) → **bind** (записать nodeName обратно в api-server). Смотрит на **requests**, не limits. С kubelet напрямую НЕ говорит.
- 4 **④ kubelet.** На ноде watch'ит: «есть под с МОИМ nodeName?». Увидел — поднимает через триаду: **CRI** (cri-o скачивает образ, запускает контейнеры, ID `cri-o://...`) + **CNI** (Calico даёт IP и сеть) + **CSI** (монтирует volumes). Гоняет пробы → пишет `status`: Running, потом Ready.
- 5 **⑤ всё через api-server.** Никто не разговаривает напрямую — только через api-server, а `etcd` это память за ним. Та же reconcile-машина и самовосстанавливает: умер под → ReplicaSet-controller увидит расхождение → цепочка прокрутится заново.

Задача: не поверить на слово, а проследить цепочку вживую — увидеть, как из одной команды рождаются ReplicaSet, Pod-объекты и как у пода **появляется** nodeName.

CLUSTER прослеживаем цепочку apply вживую

```

kubectl apply -f deploy.yaml           # отправили desired state в api-server

# 1) что кластер делал по шагам – события по времени
kubectl get events --sort-by=.lastTimestamp | tail

# 2) Deployment-controller уже создал ReplicaSet?
kubectl get rs

# 3) у подов ПОЯВИЛАСЬ нода (scheduler отработал bind)
kubectl get pods -o wide               # смотри колонку NODE – была пустой

# 4) тайминг событий конкретного пода: Scheduled → Pulled → Started
kubectl describe pod <p> | grep -A5 Events

```

ОЖИДАЕМЫЙ ВЫВОД

Scheduled	default-scheduler	Successfully assigned default/web-xxxx to node3
Pulling/Pulled	kubelet	cri-o скачал образ
Created/Started	kubelet	контейнер запущен → Running → Ready

Читаем вывод: в `events` видно ту самую цепочку по времени. До scheduler в `get pods -o wide` колонка NODE пустая, после bind – там `node3`. В `describe` событие **Scheduled** – это scheduler, **Pulled/Started** – это уже kubelet. Никакого «магического запуска» – каждый шаг оставил след.

СОБЕС

«Что происходит при `kubectl apply`?» Правильный ответ – **вся цепочка**: kubectl → api-server (authn→authz→admission→validate→etcd) → controller-manager (Deployment→ReplicaSet→Pod) → scheduler (filter→score→bind) → kubelet (CRI+CNI+CSI, пробы, status). А не «создался под». И всё общение – только через api-server.



Пантеон: компоненты k8s = боги Олимпа

Все компоненты мы уже разобрали по отдельности. Теперь — собираем героев перед финальной сценой. Если бы Kubernetes жил в Древней Греции, его компоненты возвели бы в богов. Эта мнемоника намертво раскладывает «кто за что» — а на собесе ты не путаешь scheduler с controller-manager.

БОГ	КОМПОНЕНТ	ЗОНА ОТВЕТСТВЕННОСТИ
Зевс	API Server	царь — ВСЕ запросы только через него, проверяет права (authn/authz/admission)
Мнемосина	etcd	богиня памяти — помнит всё состояние кластера, в нескольких копиях (кворум)
Афина	Scheduler	стратег — решает, на какую ноду поставить Pod (filter → score → bind)
Фемида	Controller Manager	держит закон «да будет 5 реплик» — пал Pod, создаёт нового
Арес	Kubelet	бог войны на каждой ноде — исполнитель: создать / уничтожить контейнер
Гермес	kube-proxy	посланник — ЗАРАНЕЕ прокладывает дороги (сетевые правила); сам трафик не через него
 Прометей	DevOps + ИИ	принёс смертным огонь — это ты со связкой практики и нейросети

Мнемоника: Зевс пропускает приказы и хранит порядок (**API**), Мнемосина всё помнит (**etcd**), Афина думает куда (**scheduler**), Фемида следит, чтобы закон соблюдался (**controllers**), Арес идёт и делает (**kubelet**), Гермес заранее проложил дороги (**kube-proxy**). Прочитал слева направо — и у тебя в голове весь apply.

✓ Ключевой развод на собесе

Гермес (kube-proxy) **заранее прокладывает дороги** — настраивает iptables/IPVS-правила. Сам пакет с трафиком через kube-proxy НЕ идёт: он летит по правилам, которые тот уже разложил. Путаница «трафик идёт через kube-proxy» — классический минус на интервью.

СОБЕС

«Назови компоненты control-plane и кто за что» — раскладывай через пантеон: Зевс=API Server (входная дверь и права), Мнемосина=etcd (память), Афина=Scheduler (куда), Фемида=Controller Manager (желаемое=реальное), Арес=Kubelet (исполнитель на ноде), Гермес=kube-proxy (сетевые правила заранее). Чёткое разделение ролей = чёткий ответ.



Итог Блока 1 + что дальше в серии

Прошли блок не по картинкам, а руками. Разобрали **зачем вообще нужен Kubernetes** (декларативность, самовосстановление, оркестрация), его **архитектуру** (control-plane vs ноды, кто за что отвечает), ключевые **сущности** (Pod, Deployment, Service, ConfigMap/Secret, PV/PVC), прошли **цепочку apply** по атомам — и подняли **свое приложение на своём кластере** из 3 VM через kubespray (cri-o + Calico). Это не теория из ролика — это твой живой стенд.

i Путь трафика — финальная картинка

Снаружи внутрь: **внешний вход** → **NodePort** → **kube-proxy (правила DNAT)** → **Pod**. Внутри кластера: **Service (ClusterIP)** → **endpoints** → **Pod**. Сервис не «держит» поды — он по labels собирает их адреса в endpoints, а kube-proxy заранее разложил правила, по которым пакет долетает до конкретного пода. Это мы и разберём дальше по атомам.

Дальше в серии — по нарастающей

- ① **Путь трафика на атомы** — kube-proxy в режиме iptables/IPVS, DNAT, как CNI/Calico доводит пакет до пода. Разберём каждый прыжок пакета.
- ② **Отказоустойчивость + балансировщики** — почему один control-plane это риск; HAProxy перед api-server, VIP через keepalived, MetalLB для on-prem LoadBalancer.
- ③ **Боевой HA-кластер** — kubespray вглубь + прод-обвязка: логи, мониторинг, безопасность, бэкап etcd. То, что отличает «поднял дома» от «держу в проде».
- ④ **CI/CD и GitOps** — Docker → CI → Helm → ArgoCD. Как код доезжает до кластера сам, без ручного `kubectl apply`.

ИТОГ

Куб — **не магия**. Мы разобрали его по атомам: от «зачем» до живого кластера на своих VM и цепочки apply по шагам. Дальше — глубже в трафик, отказоустойчивость и доставку. У тебя уже есть стенд, на котором можно потрогать каждую следующую тему руками.



kube-proxy под микроскопом: Service — это просто правила

Запомни главное: **kube-proxy не возит трафик**. Он не сидит в середине и не проксирует пакеты (это легаси-режим `userspace`, его давно нет). Он — **контроллер, который пишет правила ядра**. Слушает `api-server`, видит `Service` и его `EndpointSlice`, и переписывает таблицу `iptables` (или настраивает `IPVS`) так, чтобы ядро само подменяло адрес. `ClusterIP` живёт ТОЛЬКО в этих правилах — физически такого адреса ни на одном интерфейсе нет.

Как пакет на `ClusterIP:80` превращается в `PodIP:8080`

- 1 Под делает `curl http://10.96.0.10:80`. Пакет с `dst 10.96.0.10:80` (`ClusterIP`) уходит в сетевой стек ноды.
- 2 В таблице `nat`, цепочка `PREROUTING/OUTPUT` → прыжок в `KUBE-SERVICES` — главную точку входа `kube-proxy`.
- 3 `KUBE-SERVICES` матчит пару (`ClusterIP` + порт) и прыгает в персональную цепочку сервиса `KUBE-SVC-<hash>`. Хэш — от имени сервиса+порта, отсюда «нечитаемые» имена.
- 4 `KUBE-SVC-<hash>` = балансировщик. Через модуль `statistic --mode random --probability` с вероятностью `1/N` выбирает один из `KUBE-SEP-<hash>` (`SEP` = `Service EndPoint`, по одному на живой под).
- 5 `KUBE-SEP-<hash>` делает `DNAT`: подменяет `dst`-адрес `10.96.0.10:80 → 10.233.71.4:8080` (реальный `PodIP:targetPort`).
- 6 Ядро через `conntrack` запоминает связку, чтобы ответный пакет прошёл обратный путь (`un-DNAT`). Дальше — обычный роутинг до пода (`CNI/Calico`).

NODE видим цепочки `kube-proxy` своими глазами

```
iptables -t nat -L KUBE-SERVICES -n | head      # точка входа: список всех ClusterIP
iptables -t nat -L KUBE-SVC-XXXX -n           # балансировка по probability
iptables -t nat -L KUBE-SEP-XXXX -n           # строка DNAT to:PodIP:port
conntrack -L | grep 10.96.0.10                # живые связки после трафика
```

ЧТО ИСКАТЬ

```
DNAT tcp -- 0.0.0.0/0 10.96.0.10 tcp dpt:80 to:10.233.71.4:8080
```

i Почему `ClusterIP` не пингуется

`ping ClusterIP` почти всегда молчит — и это норма. `ClusterIP` виртуальный: он существует лишь как условие `--dst` в правилах `nat`, привязанных к **TCP/UDP-портам** сервиса. `ICMP` (`ping`) не имеет порта → ни одно правило не матчит → `DNAT` не происходит → отвечать некому. Проверь сервис через `curl` на его порт, а не пингом.

✓ IPVS vs iptables — почему на масштабе берут IPVS

iptables-правила ядро проходит **линейным списком**: цепочка KUBE-SERVICES при 5000 сервисов = тысячи правил, которые сканируются по очереди — $O(n)$. IPVS держит бэкенды в **хэш-таблице** в ядре: поиск назначения — $O(1)$, плюс настоящие алгоритмы балансировки (rr, lc, wrr). До ~1000 сервисов разницы не чувствуешь; на крупных кластерах iptables начинает добавлять заметную задержку на установку соединения — тогда переключают режим на IPVS.

СОБЕС

«kube-проху ничего не проксирует — он контроллер, который из EndpointSlice генерит правила. В режиме iptables: KUBE-SERVICES → KUBE-SVC (пандом-балансировка) → KUBE-SEP (DNAT ClusterIP→PodIP). ClusterIP виртуальный, живёт только в nat-таблице, поэтому не пингуется. На масштабе iptables — $O(n)$, IPVS — хэш-таблица $O(1)$ ».



CNI и VXLAN по байтам: как pod-пакет долетает до другой ноды

Kubernetes сам сеть не умеет — он делегирует. **CNI (Container Network Interface)** — это контракт: набор **бинарников** в `/opt/cni/bin/`. Когда kubelet поднимает под, он вызывает плагин с командой **ADD** (создать пара-интерфейс, выдать IP, прописать маршруты); при удалении — **DEL**. Общение через stdin (JSON-конфиг) и переменные окружения, никакого демона между ними.

Кто за что отвечает в Calico

- 1 **IPAM** (IP Address Management): нарезает PodCIDR кластера на **блоки /26** (64 адреса) и закрепляет блок за нодой. Поэтому поды одной ноды лежат в соседних адресах — это её блок.
- 2 **CNI-плагин calico** при ADD: создаёт `veth`-пару (виртуальный кабель), один конец — `eth0` внутри пода, второй — `caliXXXX` в неймспейсе ноды.
- 3 **Felix** (агент Calico на каждой ноде) — мозг датаплейна: программирует маршруты в таблице роутинга ноды и правила фильтрации (NetworkPolicy) в iptables.
- 4 Felix узнаёт, какой блок /26 на какой ноде живёт, и ставит маршрут: «адреса блока ноды-В → отправлять через интерфейс `vxlan.calico` на адрес ноды-В».

VXLAN-инкапсуляция: пакет в пакете (конверт в конверте)

Поды на разных нодах в разных подсетях /26 — физсетью про PodIP ничего не знает. Решение: **обернуть** оригинальный пакет так, будто это обычный UDP между нодами. Felix формирует слои снаружи внутрь:

- 1 **Внешний IP-заголовок**: src = NodeIP отправителя, dst = NodeIP получателя. Это видит физическая сеть — для неё летит обычный трафик ноды↔нода.
- 2 **UDP-заголовок, порт назначения 4789** — стандартный порт VXLAN. По нему принимающее ядро понимает: внутри VXLAN, надо распаковать.
- 3 **VXLAN-заголовок с VNI** (VXLAN Network Identifier) — метка виртуальной сети, отделяет оверлей от чужого трафика на 4789.
- 4 **Внутри — оригинальный Ethernet+IP-кадр пода**: src = PodIP, dst = PodIP. На приёмной ноде ядро снимает три внешних слоя и кладёт исходный пакет в её сеть → он доезжает до нужного `cali`-интерфейса.

NODE смотрим маршруты оверлея и инкапсуляцию

```
ip route | grep vxlan.calico      # блоки /26 чужих нод → через vxlan.calico
ip -d link show vxlan.calico      # vxlan id (VNI), dstport 4789
tcpdump -ni eth0 udp port 4789    # ловим инкапсулированные пакеты между нодами
```

i BGP-native vs VXLAN — когда что

VXLAN (оверлей): поверх любой физсети, ноды просто должны видеть друг друга по IP. Цена — лишние ~50 байт заголовков на каждый пакет и работа ядра на упаковку/распаковку. Берут, когда сеть «чужая»: облако, нет доступа к роутерам, ноды в разных подсетях. **BGP-native**: Calico через протокол BGP анонсирует реальные PodCIDR соседям/роутерам — пакет пода идёт по сети как есть, БЕЗ обёртки, с настоящим PodIP. Быстрее и проще для трейсинга, но требует, чтобы сеть/роутеры умели BGP (свой дата-центр, bare-metal). Правило: контролируешь сеть → BGP; не контролируешь → VXLAN.

🔊 Грабля: MTU и фрагментация

Обёртка съедает ~50 байт. Если внутри пода MTU=1500, а физсеть тоже 1500 — обёрнутый пакет станет 1550 и будет резаться/тормозить. Calico занижает MTU на `vxlan.calico` (обычно 1450). Симптом неправильного MTU: `ping` и мелкие запросы идут, а крупные ответы (HTTPS, большие тела) «висят».

СОБЕС

«CNI — это бинарники, kubelet зовёт их ADD/DEL. Calico: IPAM нарезает /26 на ноду, плагин делает veth-пару, Felix пишет маршруты. В VXLAN-режиме pod-пакет инкапсулируется: внешний IP нод + UDP:4789 + VXLAN-VNI + оригинальный кадр пода. BGP-native шлёт реальный PodIP без обёртки, но нужна сеть с BGP».



Scheduler изнутри: filter → score → bind

kube-scheduler не «запускает» поды — он только **решает, на какую ноду** их посадить, и записывает решение. Под создаётся со ПУСТЫМ `spec.nodeName`; scheduler берёт такие поды из очереди и прогоняет через два цикла плагинов — сначала **отсеять негодные ноды** (filter), потом **оценить годные** (score). Запускает контейнеры уже kubelet на выбранной ноде — это другой процесс.

Полный цикл одного пода

- 1 **Ⓢ FILTER (предикаты, бинарно «да/нет»)**: из всех нод выкидываем те, где под физически не поместится или не имеет права быть.
- 2 **NodeResourcesFit** — хватает ли свободных CPU/RAM под **requests** пода (именно requests, не limits). **NodeAffinity** — соответствует ли нода `nodeSelector` /affinity-правилам. **TaintToleration** — есть ли у пода toleration на taint ноды. **PodTopologySpread** — не нарушит ли размещение правила равномерности по зонам. **VolumeBinding** — доступен ли требуемый том в этой зоне.
- 3 **Ⓢ SCORE (приоритизация, 0-100)**: оставшиеся ноды оцениваем и ранжируем. **LeastAllocated** — выше балл менее загруженной (размазать нагрузку) ИЛИ **MostAllocated** — наоборот плотно набить (экономия нод). **ImageLocality** — плюс ноде, где образ уже скачан (быстрый старт). **InterPodAffinity** — плюс за близость/удалённость к нужным подам.
- 4 **Ⓢ BIND**: нода-победитель выбрана. Scheduler делает один вызов api-server — записывает `spec.nodeName=<нода>` в объект пода. Всё, его работа закончена.
- 5 **kubelet** этой ноды видит «появился под с моим nodeName», зовёт CRI (cri-o) и CNI → контейнеры стартуют.

CLUSTER наблюдаем решение шедулера

```
kubectl get pod <pod> -o wide # колонка NODE — куда сел
kubectl describe pod <pod> | grep -A3 Events # Scheduled / FailedScheduling + причина
kubectl get events --field-selector reason=FailedScheduling
```

ТИПИЧНЫЙ ОТКАЗ

```
0/3 nodes available: 3 Insufficient cpu, 1 node(s) had untolerated taint
```

Почему requests, а не limits

Scheduler считает «вместимость» ноды по сумме **requests** всех её подов, а не limits. Логика: requests — это **гарантия**, бронь ресурсов, которую под получит точно. limits — лишь потолок «не больше чем», и поды редко упираются в него одновременно. Если бы считали по limits, ноды простаивали бы полупустыми (бронировали бы пики, которых нет). Отсюда практический вывод: ставь requests честно — заниженные приводят к переподписке и борьбе за ресурсы, завышенные оставляют ноду пустой.

✓ Preemption — вытеснение ради приоритета

Если под с высоким `priorityClassName` никуда не влезает (filter отсекал все ноды по ресурсам), включается **preemption**: scheduler ищет ноду, где, выселив один-два пода **пониже приоритетом**, он освободит место. Жертвам отправляется graceful-termination, после чего высокоприоритетный садится. Так критичные ворклоады не стоят в очереди за фоновыми задачами.

🔧 «Почему Pending» = filter не нашёл ноду

Под в `Pending` почти всегда означает: **стадия filter отсеяла все ноды**. Читай Events дословно — там перечислено, сколько нод и по какой причине отпало: `Insufficient cpu/memory` (мало ресурсов под requests), `untolerated taint` (нет toleration), `node affinity mismatch` (не подходит по selector), `didn't match pod topology spread`. Это не «кластер сломан» — это scheduler честно объясняет, почему некуда сажать.

СОБЕС

«Scheduler работает в два цикла: filter отсеивает ноды предикатами (NodeResourcesFit по requests, NodeAffinity, TaintToleration, TopologySpread, VolumeBinding), score ранжирует выживших (LeastAllocated, ImageLocality, InterPodAffinity), bind пишет nodeName. Считает по requests, потому что это гарантия. Pending = filter не нашёл ноду — смотрю Events. Если высокоприоритетный не влез — preemption выселяет низкоприоритетных».



etcd и Raft: как кластер договаривается об истине

etcd — это распределённое key-value хранилище, единственный источник правды кластера. «Распределённое» значит: одни и те же данные на 3 нодах, и им нужно **согласие**, что именно записано, даже если кто-то упал. Эту задачу решает алгоритм консенсуса **Raft**. Идея Raft: один **лидер** на текущий период, остальные — **последователи**, повторяющие его журнал. Запись считается реальной, только когда её подтвердило **большинство**.

Жизнь записи через Raft

- 1 Term (терм)** — счётчик «эпох». Каждые выборы повышают term на 1. Любое сообщение несёт term; узел с устаревшим term всегда уступает более свежему — так отсекаются «зомби»-лидеры.
- 2 Leader election:** если follower не слышит лидера за timeout, он становится **candidate**, повышает term и шлёт **RequestVote** остальным. Набрал большинство голосов → стал лидером. Голос — один на term, поэтому два лидера в одном term невозможны.
- 3 Реплицируемый лог:** клиент (api-server) пишет только лидеру. Лидер добавляет запись в свой журнал и шлёт **AppendEntries** последователям с этой записью.
- 4 Commit:** как только **большинство** подтвердило запись в свой журнал, лидер двигает **commit index** — запись считается зафиксированной и применяется к состоянию. Только теперь api-server получает «ОК».
- 5 Heartbeat:** пустой **AppendEntries** лидер шлёт постоянно — он же сигнал «я жив, выборы не нужны». Пропал heartbeat → followers начинают новые выборы.

Почему 3 ноды переживают падение одной, но не двух

- 1 Кворум = большинство = $N/2+1$.** Для 3 нод кворум = 2. Любая запись/выбор требует согласия минимум 2 узлов.
- 2 Упала 1 нода из 3 → живых 2 → $2 \geq 2$ → кворум есть → кластер пишет и выбирает лидера. Работает.**
- 3 Из задачки «5 нод, 3 упали» → живых 2, а кворум для 5 = 3. $2 < 3$ → кворума нет → etcd уходит в read-only / встаёт, api-server не может писать. Кластер «застыл» не потому что сломался, а потому что отказывается принять решение без большинства.**
- 4 Split-brain (раскол сети):** кластер разделился на 2+3. Группа из 3 имеет большинство → выбирает лидера и работает. Группа из 2 большинства не имеет → не выбирает лидера, не пишет. Так Raft **конструктивно не допускает двух пишущих лидеров** — меньшинство само себя блокирует.

i MVCC: ничего не затирается, всё — версии

etcd хранит данные как **MVCC** (multi-version concurrency control): каждая запись/изменение получает глобальный **revision** — монотонно растущий номер версии. Старые значения ключа не стираются сразу, лежат как история — поэтому возможен **watch с конкретного revision** и чтение «как было». Чтобы база не пухла бесконечно, периодически идёт **compaction** — выкидывание ревизий старше порога, и **defrag** — возврат места на диск.

✓ Как api-server использует etcd (watch и resourceVersion)

В etcd напрямую не лезет никто, кроме api-server. Контроллеры и kubelet не опрашивают etcd в цикле — они открывают **watch** к api-server (поверх gRPC-стрима), и тот толкает изменения по мере появления.

resourceVersion в каждом объекте Kubernetes — это и есть etcd-revision: по нему watch продолжается «с того места, где прервался», а оптимистичные апдейты (**update** с конфликтом) ловят ситуацию «объект уже поменяли» сравнением resourceVersion.

СОБЕС

«etcd согласует данные через Raft: один лидер на term, RequestVote на выборах, AppendEntries реплицирует лог, запись коммитится при подтверждении большинством (кворум $N/2+1$). 5 нод, 3 упали → $2 < 3$, кворума нет → кластер read-only. Split-brain невозможен: меньшинство не выберет лидера. Внутри — MVCC с revision (= resourceVersion), api-server раздает изменения через watch по gRPC».



Путь запроса: authn → authz → admission

Любой `kubectl apply` — это HTTP-запрос в api-server, и прежде чем что-то попадёт в etcd, он проходит три шлагбаума в строгом порядке. Перепутать их — частая ошибка на собесе. **authn** отвечает «кто ты», **authz** — «можно ли тебе это действие», **admission** — «а теперь я могу изменить или зарубить сам объект». Провал на любом этапе — запрос дальше не идёт.

Конвейер по шагам

- 1 **Ⓢ AUTHN (аутентификация) — кто ты?** api-server по очереди пробует методы, пока один не опознает: **x509 client cert** (CN=имя, O=группы), **bearer token**, **ServiceAccount JWT** (токен пода), **OIDC** (внешний провайдер, корпоративный SSO). Опознал → есть username+groups. Не опознал ни один → `401 Unauthorized`.
- 2 **Ⓢ AUTHZ (авторизация) — что тебе можно?** Берётся (username+groups) из шага 1 и проверяется право на конкретное действие (verb get/list/create + ресурс + namespace). Модули: **RBAC** (Role/ClusterRole + Binding; работает по принципу **allow-only** — по умолчанию запрещено, разрешения только добавляют), **Node authorizer** (особые права kubelet), **Webhook** (внешний сервис решает). Нет разрешающего правила → `403 Forbidden`.
- 3 **Ⓢ ADMISSION — могу ли я тронуть сам объект?** Двухфазно, и порядок жёсткий:
- 4 **3a. Mutating (изменяющие)** — могут **дописать/поменять** объект: SA-инжектор подкладывает токен и default ServiceAccount, **LimitRanger** проставляет default requests/limits, sidecar-инжекторы добавляют контейнеры.
- 5 **3b. Schema validation** — между фазами объект проверяется на соответствие схеме (OpenAPI): валидный ли YAML, есть ли обязательные поля.
- 6 **3c. Validating (проверяющие)** — только говорят **да/нет**, объект уже не меняют: **PodSecurity** (не привилегированный ли), **ResourceQuota** (не превышен ли лимит namespace), кастомные ValidatingWebhook. Прошёл → объект пишется в etcd.

CLUSTER проверяем каждый шлагбаум

```
kubectl auth whoami           # authn: за кого меня держит api-server
kubectl auth can-i create pods -n dev  # authz: разрешено ли (RBAC)
kubectl auth can-i --list      # все мои разрешения
kubectl get validatingwebhookconfigurations  # admission: кто проверяет
kubectl get mutatingwebhookconfigurations    # admission: кто дописывает
```

Почему mutating строго ДО validating

Порядок не случаен — иначе система противоречила бы сама себе. Mutating-фаза **дописывает** объект: проставляет default requests, инжектит sidecar, подкладывает SA-токен. Validating-фаза **выносит финальный приговор** и объект уже не трогает. Если бы validating шёл раньше — он проверял бы **черновик**, а потом mutating добавил бы что-то непроверенное (например, sidecar, нарушающий PodSecurity), и оно уехало бы в etcd мимо контроля. Поэтому: сначала собрали финальный вид объекта (mutating), затем проверили именно его (validating). Внутри каждой фазы тоже есть порядок — он задаётся через reinvocation, чтобы один мутатор увидел правки другого.

Грабля: 401 vs 403 — не путать

401 Unauthorized = провал на **authn**: api-server не понял, КТО ты (протух/неверный токен, нет сертификата). **403 Forbidden** = ты опознан, но провал на **authz**: КОМУ ты — известно, а вот ПРАВ на это действие нет (RBAC не дал). Лечатся по-разному: 401 — чинишь credentials, 403 — добавляешь Role/RoleBinding.

СОБЕС

«Запрос в api-server идёт **authn** → **authz** → **admission**. **authn** = кто ты (x509 / token / SA JWT / OIDC), провал = 401. **authz** = что можно (RBAC allow-only, Node, Webhook), провал = 403. **admission** двухфазно: сначала **Mutating** (дописывает — LimitRanger, SA-инжектор), потом **schema validation**, потом **Validating** (да/нет — PodSecurity, ResourceQuota). **Mutating** строго до **Validating**, чтобы проверяли финальный объект, а не черновик».



Образ изнутри: слои, overlayfs и почему кэш работает

Образ контейнера (OCI image) — это не один файл, а **три вещи**: **manifest** (список из чего собран + дайджесты), **config** (env, entrypoint, на каком слое что), и сами **слои** — tar-архивы файловой системы. Ключевая идея: всё **content-addressable** — адресуется по **sha256** своего содержимого. Поменялся хоть байт в слое → новый sha256 → это уже другой слой. Отсюда растут и кэш, и шаринг.

Как образ становится работающей ФС: overlayfs

- 1 lowerdir** — слои образа, смонтированные **только на чтение** (readonly), сложенные стопкой. Это «база», общая для всех контейнеров из этого образа.
- 2 upperdir** — единственный **writable**-слой, свой у каждого запущенного контейнера. Всё, что контейнер пишет/меняет в рантайме, оседает здесь.
- 3 merged** — итоговая точка монтирования, которую видит процесс внутри контейнера: ядро **накладывает** upperdir поверх lowerdir и показывает их как одну ФС.
- 4 copy-up при записи**: контейнер хочет изменить файл из readonly-слоя → ядро сначала **копирует** его в upperdir и правит уже копию. Оригинал в слое нетронут. Поэтому старт мгновенный — слои не копируются, копируется только то, что реально меняешь.
- 5 whiteout при удалении**: нельзя удалить файл из readonly-lowerdir. Вместо этого в upperdir создаётся спец-маркер **whiteout** — он «прячет» файл, и в merged его не видно, хотя в нижнем слое он лежит.

NODE смотрим слои и overlay-монтирование

```
crictl images          # образы на нодe по digest
mount | grep overlay   # lowerdir=.../l1:.../l2 upperdir=... merged
ls /var/lib/containers/storage/overlay/ # каталоги слоёв (cri-o)
```

✓ Почему слои кэшируются и шарятся → COPY go.mod ДО COPY ..

Каждая инструкция Dockerfile = слой с sha256. При пересборке демон **переиспользует** слой, если входные данные не изменились. Отсюда правило: **сначала** копируй только манифест зависимостей и ставь их, и лишь **потом** весь код:

① **COPY go.mod go.sum .** → ② **RUN go mod download** → ③ **COPY . .** → ④ **RUN go build**

Поправил строчку в коде → меняется только слой ③ и ниже, а тяжёлый слой ② (скачанные зависимости) берётся из кэша. Сделай наоборот (**COPY . .** первым) — любая правка кода инвалидирует слой и зависимости качаются заново каждую сборку. Плюс шаринг: одинаковый по sha256 слой хранится на нодe **один раз** и переиспользуется всеми образами/контейнерами.

i Registry: tag — это ярлык, digest — это истина

В реестре образ адресуется двумя способами. **tag** (**:latest**, **:1.2**) — подвижная метка, её можно перевесить на другой образ. **digest** (**@sha256: ...**) — неизменяемый адрес содержимого: один и тот же digest = байт-в-байт один образ, всегда. Для повторяемости в проде пинят по digest, а не по тегу. **Manifest list (image index)** решает multi-arch: один тег указывает на список манифестов под разные платформы (amd64, arm64), и рантайм сам тянет манифест под свою архитектуру.

Как cri-o разворачивает образ при запуске пода

- 1 **pull** — по тегу/дайджесту тянет manifest, по нему — config и недостающие слои (уже имеющиеся по sha256 не качает).
- 2 **unpack** — распаковывает tar-слои в каталоги storage, проверяя sha256 каждого.
- 3 **overlay mount** — собирает lowerdir из слоёв, добавляет свежий upperdir для контейнера, монтирует merged → отдаёт как rootfs процессу.

СОБЕС

«Образ = manifest + config + слои, всё content-addressable по sha256. На ноде он раскрывается через overlays: lowerdir (readonly-слои) + upperdir (writable-слой контейнера) = merged; запись делает copy-up, удаление — whiteout. Слои кэшируются и шарятся по sha256 — поэтому COPY go.mod и go mod download идут ДО COPY . ., чтобы правка кода не пересобирала зависимости. В registry tag подвижен, digest неизменяем; manifest list даёт multi-arch».



namespaces и cgroups: из чего на самом деле сделан контейнер

«Контейнер» — не сущность ядра, а обычный процесс Linux, которому ядро **наврало про окружение** двумя механизмами. **namespaces** отвечают за **изоляция видимости** («что процесс видит» — свои PID, свою сеть, свою ФС). **cgroups** — за **ограничение ресурсов** («сколько процессу можно» — CPU, память, IO). Запомни связку: *namespaces = что вижу, cgroups = сколько могу*.

7 типов namespaces — что именно изолирует каждый

NAMESPACE	ЧТО ИЗОЛИРУЕТ / ДАЁТ «СВОЙ»
pid	дерево процессов: внутри свой PID 1, чужих процессов не видно
net	сетевой стек: свои интерфейсы, IP, порты, таблица маршрутов, iptables
mnt	точки монтирования: своя файловая иерархия, свой <code>/</code>
uts	hostname и domainname — свои, можно менять независимо от хоста
ipc	System V IPC / POSIX-очереди и shared memory между процессами
user	маппинг UID/GID: root (uid 0) внутри $\neq$ root на хосте — основа rootless
cgroup	скрывает реальную cgroup-иерархию хоста, показывает «корнем» свою

i cgroups v2: чем реально режут ресурсы

Современный единый интерфейс (одна иерархия вместо россыпи контроллеров v1). Ключевые файлы: **CPU** — `cpu.max` (жёсткий потолок «N мкс за период», это limit) и `cpu.weight` (доля при конкуренции, мягкий приоритет, это request); **память** — `memory.max` (жёсткий потолок RAM); **диск** — `io.max` (лимит IOPS/throughput). Превысить `cpu.max` нельзя — процесс притормозят; превысить `memory.max` нельзя — процесс убьют.

Как requests/limits Kubernetes превращаются в cgroup-значения

- 1 memory limit → memory.max:** жёсткий потолок. Дошёл до него — ядро не даёт больше памяти, срабатывает OOM-killer.
- 2 cpu limit → cpu.max:** жёсткий потолок процессорного времени за период (троттлинг при достижении).
- 3 cpu request → cpu.weight:** НЕ потолок, а **доля при конкуренции**. Если CPU свободен — под берёт сколько хочет; если все дерутся за ядра — request задаёт пропорцию дележа.
- 4 memory request** cgroup-лимитом не становится — это «броня» для шедулера (см. раздел про scheduler), а не ограничение в ядре.

 OOMKilled (137) vs троттлинг — RAM убивает, CPU тормозит

Это асимметрия, на которой валятся на собесе. **Память — несжимаемый ресурс**: упёрся в `memory.max` → процесс мгновенно **убивают**, под получает статус `OOMKilled` и код выхода **137** (= 128 + сигнал 9/SIGKILL). Никакого «подождать» — память нельзя «занять в долг». **CPU — сжимаемый ресурс**: упёрся в `cpu.max` → процесс не убивают, его просто **троттлят** (тормозят, заставляют ждать кванты времени). Симптомы разные: 137 в рестартах = мало памяти (поднимай memory limit); высокая latency без рестартов = CPU-троттлинг (поднимай cpu limit или оптимизируй).

✓ pause-контейнер: почему контейнеры пода видят общий localhost

В каждом поде первым стартует невидимый крошечный **pause-контейнер** («песочница»). Его единственная работа — **держат неймспейсы** пода, прежде всего `net` и `ipc`. Все «настоящие» контейнеры пода стартуют не со своим net-namespace, а **джойнятся в namespace pause-контейнера**. Отсюда два факта, которые спрашивают: контейнеры одного пода **(а) делят один IP пода и (б) видят друг друга по localhost** (общий net-namespace = общий loopback). При этом `mnt` и `pid` у них обычно свои — файловые системы и процессы разделены. pause живёт всё время пода: контейнеры можно перезапускать, а namespace и IP остаются стабильными, потому что их держит он.

СОБЕС

«Контейнер = процесс + namespaces (изоляция видимости: pid/net/mnt/uts/ipc/user/cgroup) + cgroups (ограничение ресурсов). k8s маппит: memory limit → memory.max, cpu limit → cpu.max, cpu request → cpu.weight (доля при конкуренции), memory request — только бронь шедулера. Память несжимаема: превышение → OOMKilled, код 137; CPU сжимаем: превышение → троттлинг. pause-контейнер держит net+ipc namespace, остальные контейнеры пода джойнятся → общий Pod IP и localhost».



Pod: probes, resources, env, тома

Pod — наименьшая единица, которую запускает Kubernetes. Один или несколько контейнеров с общими сетью и томами. На собесе спрашивают не «что такое pod», а как настроить **пробы**, **лимиты** и **проброс конфигов** — разбираем построчно.

LOCAL pod.yaml

```

apiVersion: v1
kind: Pod
metadata:
  name: scott
  namespace: dev          # в каком неймспейсе живёт под
  labels:
    app: scott            # по этой метке Service найдёт под
spec:
  containers:
    - name: app
      image: <registry>/scott-pilgrim-quotes:1.0.0
      ports:
        - containerPort: 8080 # порт, который слушает процесс
      env:                      # переменные окружения
        - name: APP_MODE
          valueFrom:
            configMapKeyRef:    # значение тянем из ConfigMap
              name: scott-config
              key: mode
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:      # а это — из Secret (не в открытом виде)
              name: scott-secret
              key: db_password
      resources:
        requests:              # СКОЛЬКО гарантируем (по этому планируется на ноду)
          cpu: "100m"          # 0.1 ядра
          memory: "128Mi"
        limits:                 # ПОТОЛОК (выше memory → OOMKilled)
          cpu: "500m"
          memory: "256Mi"
      startupProbe:             # «приложение вообще запустилось?» — даёт время на старт
        httpGet: { path: /healthz, port: 8080 }
        failureThreshold: 30
        periodSeconds: 2       # 30×2 = до 60с на медленный старт
      readinessProbe:           # «готов принимать трафик?» — управляет endpoints
        httpGet: { path: /ready, port: 8080 }
        periodSeconds: 5
      livenessProbe:            # «ещё живой?» — если нет, kubelet рестартит контейнер
        httpGet: { path: /healthz, port: 8080 }
        periodSeconds: 10
      volumeMounts:            # куда внутри контейнера примонтировать том
        - name: cache
          mountPath: /var/cache/app
  volumes:                     # сами тома пода
    - name: cache
      emptyDir: {}             # временный диск, живёт пока жив под

```

Разбор ключевых полей

ПОЛЕ	ЧТО ДЕЛАЕТ / ЕСЛИ УБРАТЬ
<code>metadata.labels</code>	Метки пода. По ним Service отбирает поды в endpoints. Убрать — Service не найдёт под, трафик не пойдёт.
<code>resources.requests</code>	Гарантированный минимум; по нему scheduler выбирает ноду. Убрать — под влезет куда угодно, при нехватке памяти его выселят первым.
<code>resources.limits</code>	Жёсткий потолок. Превышение memory → OOMKilled , CPU — троттлинг. Убрать — под может съесть всю ноду.
<code>startupProbe</code>	Прикрывает медленный старт: пока не прошла, liveness/readiness не работают. Убрать — liveness может убить приложение, не дав ему подняться.
<code>readinessProbe</code>	Не прошла → под убирается из endpoints (трафик не идёт), но НЕ рестартится. Убрать — трафик польётся в ещё не готовый под → 502.
<code>livenessProbe</code>	Не прошла → kubelet рестартит контейнер. Убрать — зависший процесс будет висеть «живым» вечно.
<code>secretKeyRef</code>	Тянет значение env из Secret по ключу. Опечатка в key — под не стартует: CreateContainerConfigError .
<code>volumeMounts</code> ↔ <code>volumes</code>	Имя в mount должно совпасть с volume. Рассинхрон имён — под не создастся.

CLUSTER применяем и читаем состояние

```
kubectl apply -f pod.yaml -n dev          # создать/обновить под
kubectl get pod scott -n dev -o wide      # STATUS, READY 1/1, нода, IP
kubectl describe pod scott -n dev         # главное — Conditions и Events
```

DESCRIBE → ЧТО ЧИТАТЬ

```
Conditions:
  Ready          True      # readinessProbe прошла → под в endpoints
  ContainersReady True
Events:
  Pulled        image "...scott-pilgrim-quotes:1.0.0"
  Started       container app          # старт OK
  Unhealthy     Readiness probe failed # ← если 503, смотри сюда
```

Читаем: **Conditions: Ready True** = readiness прошла, под получает трафик. В **Events** видна реальная причина: не скачался образ, упала проба, не нашёлся Secret — всё тут.

Грабля: liveness без startupProbe

Медленное приложение (миграции БД на старте) liveness убивает раньше, чем оно поднялось → бесконечный **CrashLoopBackOff**. Лечится startupProbe с запасом по времени.

СОБЕС

«Три пробы — разная ответственность: startup даёт время на старт, readiness рулит трафиком через endpoints, liveness рестартит зависший контейнер. requests — для планирования, limits — потолок; превышение memory = OOMKilled».



Deployment: реплики, стратегия, rollout

Голый Pod никто не деплоит — он не самовосстанавливается. Deployment держит **N одинаковых реплик**, обновляет их без даунтайма и умеет откатываться. Под капотом он управляет ReplicaSet, а тот — подами.

LOCAL deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: scott
  namespace: dev
spec:
  replicas: 3 # сколько подов держать ВСЕГДА
  revisionHistoryLimit: 5 # сколько старых ReplicaSet хранить для отката
  selector:
    matchLabels: # КАКИЕ поды считать своими
      app: scott
  strategy:
    type: RollingUpdate # обновлять по чуть-чуть, без простоя
    rollingUpdate:
      maxUnavailable: 1 # сколько подов МОЖНО положить во время апдейта
      maxSurge: 1 # сколько ЛИШНИХ подов поднять сверх replicas
  template: # шаблон пода (тот же spec, что в Pod)
    metadata:
      labels:
        app: scott # △ ДОЛЖЕН совпасть с selector.matchLabels
    spec:
      containers:
        - name: app
          image: <registry>/scott-pilgrim-quotes:1.0.0
          ports:
            - containerPort: 8080
          readinessProbe: # без неё RollingUpdate «слепой» — катит на неготовые
            httpGet: { path: /ready, port: 8080 }
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits: { cpu: "500m", memory: "256Mi" }
```

Разбор ключевых полей

ПОЛЕ	ЧТО ДЕЛАЕТ / ЕСЛИ УБРАТЬ
<code>replicas</code>	Желаемое число подов. Controller дотягивает факт до этого числа. Убрать — по умолчанию 1, отказоустойчивости нет.
<code>selector.matchLabels</code>	По каким меткам Deployment владеет подами. Поле иммутабельное . Не совпадёт с <code>template.labels</code> — API отвергнет манифест.
<code>template.labels</code>	Метки, которые получают поднятые поды. Должны попадать под selector, иначе Deployment «не видит» свои же поды и плодит новые.
<code>strategy.type</code>	<code>RollingUpdate</code> — без простоя; <code>Recreate</code> — сначала убить все, потом поднять (есть даунтайм). Убрать — по умолчанию RollingUpdate.
<code>maxUnavailable</code>	Сколько подов разрешено держать недоступными при апдейте. 0 — обновление медленнее, но без просадки ёмкости.
<code>maxSurge</code>	Сколько временно поднять сверх replicas. Больше surge — быстрее раскатка, но выше пик нагрузки на ноды.
<code>revisionHistoryLimit</code>	Сколько прошлых ReplicaSet хранить для <code>rollback undo</code> . 0 — откат недоступен.

CLUSTER применяем и наблюдаем раскатку

```
kubectl apply -f deployment.yaml -n dev
kubectl get deploy,rs,pods -n dev -l app=scott # deploy → rs → поды

kubectl rollout status deploy/scott -n dev # ждёт, пока докатится
kubectl rollout history deploy/scott -n dev # список ревизий для отката
# откат на прошлую версию, если новая поломана:
kubectl rollout undo deploy/scott -n dev
```

ROLLOUT STATUS → ОЖИДАЕМО

```
Waiting for deployment "scott" rollout to finish: 2 of 3 updated...
deployment "scott" successfully rolled out # все 3 пода на новой версии
```

Читаем: `successfully rolled out` = новый ReplicaSet поднял все реплики, старый ужат до 0. Если status завис — новые поды не проходят readiness; смотри `describe` пода.

і Почему важна readinessProbe в template

RollingUpdate ждёт, пока новый под станет **Ready**, и только потом гасит старый. Без пробы k8s считает под готовым сразу при старте процесса — и катит трафик в ещё не прогретое приложение.

СОБЕС «Deployment не управляет подами напрямую — он создаёт ReplicaSet под каждую ревизию. RollingUpdate с `maxUnavailable/maxSurge` гарантирует обновление без простоя, а `revisionHistoryLimit` оставляет ревизии для `rollback undo`».



Service: ClusterIP и NodePort

Поды смертны, их IP меняются. Service даёт **стабильный адрес и имя**, за которым прячется пачка подов. Он сам по меткам собирает живые поды в endpoints и балансирует трафик. ClusterIP — для доступа внутри кластера, NodePort — снаружи через порт ноды.

LOCAL service.yaml

```
# — 1) ClusterIP: внутренний адрес сервиса (по умолчанию) —
apiVersion: v1
kind: Service
metadata:
  name: scott
  namespace: dev
spec:
  type: ClusterIP          # виртуальный IP, доступен только в кластере
  selector:
    app: scott             # ловит поды с меткой app=scott → endpoints
  ports:
    - name: http
      port: 80              # на каком порту слушает SAM Service
      targetPort: 8080      # на какой порт ПОДА переслать (containerPort)
      protocol: TCP
---
# — 2) NodePort: тот же сервис, но доступен СНАРУЖИ —
apiVersion: v1
kind: Service
metadata:
  name: scott-np
  namespace: dev
spec:
  type: NodePort           # откроет порт на КАЖДОЙ ноде кластера
  selector:
    app: scott
  ports:
    - port: 80              # ClusterIP-порт (внутри тоже работает)
      targetPort: 8080      # порт пода
      nodePort: 30080       # внешний порт ноды (диапазон 30000-32767)
```

Разбор ключевых полей

ПОЛЕ	ЧТО ДЕЛАЕТ / ЕСЛИ УБРАТЬ
selector	По меткам отбирает поды в endpoints. Опечатка/убрать — endpoints пустые, Service резолвится, но трафик идти некуда (connection refused).
port	Порт, на котором слушает сам Service (его ClusterIP). По нему к сервису обращаются другие поды.
targetPort	Порт пода, куда переслать трафик (= containerPort). Не совпадёт с портом приложения — 502/refused.
nodePort	Внешний порт на всех нодах (30000–32767). Не указать — k8s выберет случайный из диапазона.
type	ClusterIP — внутри; NodePort — снаружи через IP ноды; LoadBalancer — внешний балансировщик облака. Убрать — по умолчанию ClusterIP.

CLUSTER применяем и проверяем связку selector → endpoints

```
kubectl apply -f service.yaml -n dev
kubectl get svc -n dev -o wide # TYPE, CLUSTER-IP, PORT(S)
```

```
# ГЛАВНАЯ проверка: нашёл ли Service поды по selector?
kubectl get endpointslice -n dev -l kubernetes.io/service-name=scott
kubectl describe svc scott -n dev | grep -A3 Endpoints
```

```
# NodePort снаружи (с локальной машины):
curl http://<NODE_IP>:30080/
```

ENDPOINTSlice → ОЖИДАЕМО

```
NAME          ADDRESSTYPE  PORTS      ENDPOINTS
scott-abc12    IPv4         8080       10.233.x.5,10.233.x.6,10.233.x.7
# три адреса = три живых готовых пода. Пусто → selector мимо или поды не Ready
```

Читаем: в endpointslice столько IP, сколько подов прошли readiness и попали под selector. Пустой список — самая частая причина «Service есть, а не отвечает»: метки не совпали или поды не Ready.

 **Габля:** port ≠ targetPort путают

port — куда стучатся К сервису, **targetPort** — куда он шлёт В под. Если приложение слушает 8080, а targetPort оставили 80 — будет refused, хотя Service «есть».

СОБЕС

«Service — это стабильный ClusterIP + DNS-имя; по selector он динамически собирает готовые поды в EndpointSlice, kube-proxy раскидывает на них трафик. port — порт сервиса, targetPort — порт пода, nodePort — внешний порт ноды для типа NodePort».



Ingress: домены, пути, TLS

NodePort — это «порт 30080», некрасиво и без HTTPS. Ingress — это L7-роутер: **один вход на 80/443**, дальше по домену и пути раскидывает на нужный Service. Сам манифест — лишь правила; выполняет их Ingress Controller (nginx, Traefik), который надо поставить отдельно.

LOCAL ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: scott
  namespace: dev
spec:
  ingressClassName: nginx      # КАКОЙ контроллер обрабатывает (должен быть установлен)
  tls:
    - hosts: [ scott.dev.local ]
      secretName: scott-tls    # Secret типа kubernetes.io/tls с cert+key
  rules:
    - host: scott.dev.local    # по какому домену пришли
      http:
        paths:
          - path: /            # какой путь матчим
            pathType: Prefix    # Prefix=по префиксу, Exact=точное совпадение
            backend:
              service:
                name: scott      # на КАКОЙ Service переслать
                port:
                  number: 80     # порт Service (его spec.ports.port)
```

Разбор ключевых полей

ПОЛЕ	ЧТО ДЕЛАЕТ / ЕСЛИ УБРАТЬ
<code>ingressClassName</code>	Какой контроллер исполняет правила. Убрать/нет такого класса — правило висит, но никто его не применяет, ADDRESS пустой.
<code>rules.host</code>	Домен, по которому маршрутизируем. Убрать host — правило ловит любой домен (catch-all).
<code>path + pathType</code>	<code>Prefix</code> — совпадение по началу пути; <code>Exact</code> — точное. Неверный pathType — запрос не смэтчится, уйдёт в default backend / 404.
<code>backend.service.name</code>	Имя Service-получателя в том же namespace. Опечатка — 503 от контроллера (нет апстрима).
<code>backend.service.port.number</code>	Порт Service (не пода!). Указать порт пода 8080 вместо 80 — апстрим не отвечает.
<code>tls.secretName</code>	Secret с сертификатом и ключом для HTTPS. Убрать tls — работает только по http; нет Secret — контроллер отдаёт fake-cert.

CLUSTER применяем и проверяем цепочку

```
kubectl apply -f ingress.yaml -n dev
kubectl get ingress -n dev           # должен появиться внешний ADDRESS
kubectl describe ingress scott -n dev # Rules: host/path → backend и его endpoints
```

проверка снаружи (домен резолвим на IP контроллера через /etc/hosts):

```
curl -k -H 'Host: scott.dev.local' https://<INGRESS_IP>/
```

DESCRIBE INGRESS → ЧТО ЧИТАТЬ

Rules:

Host	Path	Backends
scott.dev.local	/	scott:80 (10.233.x.5:8080,10.233.x.6:8080)
#		↑ под backend видны РЕАЛЬНЫЕ endpoints — цепочка сошлась

Читаем: под Backends в скобках стоят IP подов — значит цепочка **Ingress → Service → Pod** собралась целиком. Пусто в скобках — проблема не в Ingress, а в Service (selector/endpoints).

і Цепочка ответственности

Ingress (домен+путь → имя Service) → **Service** (имя → ClusterIP, selector → endpoints) → **Pod** (порт 8080).
Каждое звено проверяется отдельно: ingress describe → svc endpoints → pod ready.

СОБЕС

«Ingress — это L7-правила, а не процесс: их исполняет Ingress Controller, выбранный через ingressClassName. Он терминирует TLS по secretName и по host+pathType маршрутизирует на backend.service, который ведёт уже на endpoints подов».



ConfigMap и Secret: конфиг отдельно от образа

Конфиг не зашивают в образ — иначе на каждый параметр нужна пересборка. ConfigMap хранит обычные настройки, Secret — чувствительные (пароли, токены). И то и другое отдаём в под либо как **env**, либо как **файлы-том**.

LOCAL config-secret.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: scott-config
  namespace: dev
data:                                     # просто пары ключ: значение (открытым текстом)
  mode: "production"
  app.properties: |                     # многострочное значение → станет файлом в томе
    log.level=info
    quotes.limit=50
---
apiVersion: v1
kind: Secret
metadata:
  name: scott-secret
  namespace: dev
type: Opaque                             # произвольные пары ключ/значение
stringData:                             # пишем КАК ЕСТЬ, k8s сам закодирует в base64
  db_password: "S3cr3t-Pg!"
data:                                    # а здесь значения ОБЯЗАНЫ быть уже base64
  db_user: cG9zdGdyZXM=                 # echo -n postgres | base64
```

LOCAL как под их потребляет (фрагмент pod.сpec)

```
env:
  - name: APP_MODE
    valueFrom:
      configMapKeyRef: { name: scott-config, key: mode }
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef: { name: scott-secret, key: db_password }
volumeMounts:
  - { name: cfg, mountPath: /etc/app }    # app.properties ляжет файлом сюда
volumes:
  - name: cfg
    configMap: { name: scott-config }      # тот же CM, но как том
```

Разбор ключевых полей

ПОЛЕ	ЧТО ДЕЛАЕТ / ЕСЛИ УБРАТЬ
<code>ConfigMap.data</code>	Пары ключ/значение открытым текстом. Через env — фикс на момент старта; через том — обновляется в файле без рестарта.
<code>Secret.type</code> <code>Opaque</code>	Универсальный тип для своих секретов. Для TLS — <code>kubernetes.io/tls</code> , для registry — <code>dockerconfigjson</code> .
<code>stringData</code>	Значения в открытом виде — k8s сам кодирует в base64. Удобно для рук , не нужно кодировать самому.
<code>data (в Secret)</code>	Значения ОБЯЗАНЫ быть валидным base64. Положить сырой текст — ошибка <code>illegal base64</code> , объект не создается.
<code>secretKeyRef</code>	Тянет конкретный ключ Secret в env пода. Нет ключа/Secret — под застрянет в <code>CreateContainerConfigError</code> .

🔊 base64 ≠ шифрование

`base64` — это кодирование, а не защита: разворачивается одной командой. Кто видит Secret (или etcd) — видит пароль. Реальная защита: **encryption-at-rest** на api-server + RBAC на доступ к секретам + внешний Vault.

CLUSTER применяем и достаём значение (доказываем, что это не шифр)

```
kubectl apply -f config-secret.yaml -n dev
kubectl get cm scott-config -n dev -o yaml | head
kubectl get secret scott-secret -n dev          # TYPE Opaque, DATA — число ключей

# вытащить пароль: jsonpath даёт base64 → декодируем
kubectl get secret scott-secret -n dev \
  -o jsonpath='{.data.db_password}' | base64 -d ; echo
```

ОЖИДАЕМЫЙ ВЫВОД

```
S3cr3t-Pg!      # ← пароль восстановлен в открытом виде одной командой
                # именно поэтому base64 — НЕ безопасность
```

Читаем: то, что `base64 -d` мгновенно вернул пароль, и есть наглядное доказательство тезиса из ловушки — Secret по умолчанию лишь закодирован, а не зашифрован.

СОБЕС

«ConfigMap и Secret выносят конфиг из образа; подключаются как env или как том. Ключевое: Secret по умолчанию base64, а не шифр — это видно через `jsonpath | base64 -d`; для защиты нужны encryption-at-rest, RBAC и Vault».



PV, PVC, StorageClass: постоянный диск

emptyDir умирает вместе с подом — для postgres это смерть данных. Нужен постоянный том. **PVC** — заявка «дай мне 5Gi RWO»; **PV** — сам кусок диска; **StorageClass** — «фабрика», которая создаёт PV автоматически под заявку (dynamic provisioning).

LOCAL storage.yaml

```
# — StorageClass: ФАБРИКА томов (provisioner создаёт PV сам) —
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-fast
provisioner: rancher.io/local-path # КТО создаёт диск (драйвер CSI/provisioner)
reclaimPolicy: Delete             # удалить PV когда PVC удалят (Retain = оставить)
volumeBindingMode: WaitForFirstConsumer # ждать пода, чтобы выбрать ноду
---
# — PVC: ЗАЯВКА от приложения на диск —
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pg-data
  namespace: dev
spec:
  storageClassName: local-fast # из какой фабрики просить (пусто = default SC)
  accessModes: [ ReadWriteOnce ] # RW0: запись с ОДНОЙ ноды (хватает для БД)
  resources:
    requests:
      storage: 5Gi # сколько места просим
---
# — PV вручную (на оп-рем, когда нет автопроевижининга) —
apiVersion: v1
kind: PersistentVolume
metadata: { name: pv-pg-0 }
spec:
  capacity: { storage: 5Gi }
  accessModes: [ ReadWriteOnce ]
  storageClassName: local-fast
  hostPath: { path: /data/pg-0 } # демо; в проде — CSI-драйвер
```

Разбор ключевых полей

ПОЛЕ	ЧТО ДЕЛАЕТ / ЕСЛИ УБРАТЬ
<code>accessModes</code>	<code>RWO</code> — чтение/запись с одной ноды; <code>RWX</code> — с многих (нужна сетевая ФС: NFS/CephFS). Блочный диск + RWX — PVC не свяжется.
<code>requests.storage</code>	Запрашиваемый объём. PV должен быть $\geq$ этого. Просить больше, чем есть PV — PVC висит в <code>Pending</code> .
<code>storageClassName</code>	Какую фабрику просить. Пусто — берётся default SC; несуществующий класс — Pending без провижининга.
<code>provisioner</code>	Драйвер, создающий PV под заявку. Нет такого драйвера в кластере — заявка не обслуживается, PVC Pending.
<code>reclaimPolicy</code>	Судьба PV после удаления PVC: <code>Delete</code> — стереть, <code>Retain</code> — оставить данные для ручного разбора.

⚠ on-prem: PVC висит в Pending

На голом kubespray-кластере по умолчанию **нет провижинера** — динамически PV никто не создаёт. PVC будет вечно `Pending`, пока не поставишь StorageClass с драйвером (напр. local-path-provisioner) или не создашь PV руками.

CLUSTER применяем и смотрим связывание

```
kubectl apply -f storage.yaml -n dev
kubectl get sc                    # список фабрик, кто DEFAULT
kubectl get pvc -n dev            # STATUS: Pending → Bound, и к какому VOLUME
kubectl get pv                   # кем CLAIMED, RECLAIM POLICY
kubectl describe pvc pg-data -n dev # Events — почему Pending, если висит
```

GET PVC → ОЖИДАЕМО

```
NAME      STATUS  VOLUME   CAPACITY  ACCESS MODES  STORAGECLASS
pg-data   Bound   pv-pg-0   5Gi       RWO            local-fast
#         ↑ Bound = заявка получила диск. Pending → смотри describe Events
```

Читаем: **STATUS Bound** и заполненный **VOLUME** = PVC связался с PV, под сможет его примонтировать. **Pending** с **WaitForFirstConsumer** — это норма: том создастся, когда появится потребляющий под.

СОБЕС

«PVC — заявка приложения (размер + accessMode), PV — реальный том, StorageClass — фабрика для динамического создания PV через provisioner. RWO — для БД (одна нода пишет), RWX — для общего хранилища. reclaimPolicy решает, удалять ли данные с томом».



RBAC: ServiceAccount, Role, RoleBinding

«Кто что может» в кластере решает RBAC. **ServiceAccount** — личность пода. **Role** — набор разрешений (что и над чем). **RoleBinding** — связка «эта личность получает эти права». Role/RoleBinding действуют в namespace; кластерные аналоги — ClusterRole/ClusterRoleBinding.

LOCAL rbac.yaml

```
# — 1) ServiceAccount: личность для подов —
apiVersion: v1
kind: ServiceAccount
metadata:
  name: dev-reader
  namespace: dev
---
# — 2) Role: НАБОР прав внутри namespace dev —
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
rules:
  - apiGroups: [ "" ]           # "" = core-группа (pods, services, configmaps)
    resources: [ pods, pods/log ] # НАД ЧЕМ права
    verbs: [ get, list, watch ]  # ЧТО можно (только чтение)
---
# — 3) RoleBinding: ВЫДАЁМ Role этой ServiceAccount —
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dev-reader-binding
  namespace: dev
subjects:
  # КОМУ выдаём права
  - kind: ServiceAccount
    name: dev-reader
    namespace: dev
roleRef:
  # КАКУЮ роль выдаём (ссылка на Role выше)
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Разбор ключевых полей

ПОЛЕ	ЧТО ДЕЛАЕТ / ЕСЛИ УБРАТЬ
<code>rules.apiGroups</code>	В какой API-группе ресурс. <code>""</code> — core (pods, svc); <code>apps</code> — deployments. Неверная группа — правило не покрывает ресурс, доступа нет.
<code>rules.resources</code>	Над какими объектами права (pods, secrets, pods/log). Забыть pods/log — <code>kubectl logs</code> запрещён, хотя get pods разрешён.
<code>rules.verbs</code>	Какие действия: get/list/watch (чтение), create/update/patch/delete (запись). Лишний delete — нарушение least privilege.
<code>roleRef</code>	Ссылка на выдаваемую роль. Поле иммутабельное . Опечатка в name — binding ни к чему не привязан, прав нет.
<code>subjects</code>	Кому выдаём: ServiceAccount / User / Group. Забыть namespace у SA — субъект не совпадёт, права не применятся.

CLUSTER применяем и ПРОВЕРЯЕМ права от лица SA

```
kubectl apply -f rbac.yaml
kubectl get sa,role,rolebinding -n dev      # все три объекта на месте

# главное — спросить «а МОЖНО ли?» от лица сервис-аккаунта:
kubectl auth can-i get pods \
  --as=system:serviceaccount:dev:dev-reader -n dev      # должно: yes

kubectl auth can-i delete pods \
  --as=system:serviceaccount:dev:dev-reader -n dev      # должно: no

AUTH CAN-I → ОЖИДАЕМО
yes      # get pods разрешён — Role+Binding сработали
no      # delete pods запрещён — least privilege соблюдён
```

Читаем: `can-i` от лица `system:serviceaccount:<ns>:<name>` — самый быстрый способ доказать, что права именно те, что задумывались: yes на чтение, no на удаление.

і Три кита авторизации

Authentication («кто я» — SA, токен, сертификат) → **Authorization** («что мне можно» — RBAC: Role/Binding) → **Admission** («можно ли это вообще» — валидация/мутация на входе). RBAC — это средний этап.

СОБЕС

«RBAC = Role (apiGroups+resources+verbs — что над чем) + Binding (subjects ↔ roleRef — кому). Role действует в namespace, ClusterRole — глобально. Проверяю не на глаз, а через `kubectl auth can-i --as=system:serviceaccount:dev:dev-reader`».



Зоопарк: StatefulSet · DaemonSet · HPA · Job · NetworkPolicy

Когда Deployment не подходит. Кратко по мини-манифесту: **StatefulSet** — для БД (стабильные имена и диски), **DemonSet** — по поду на ноду, **HPA** — автоскейл, **Job/CronJob** — разовые/по расписанию, **NetworkPolicy** — firewall для подов.

LOCAL statefulset.yaml (postgres) + daemonset.yaml

```
kind: StatefulSet # apps/v1 · стабильные pg-0, pg-1 + свой диск каждому
spec:
  serviceName: pg-headless # headless-Service для DNS pg-0.pg-headless...
  replicas: 3
  selector: { matchLabels: { app: pg } }
  template: { metadata: { labels: { app: pg } }, /* ...спес пода... */ }
  volumeClaimTemplates: # ★ на КАЖДЫЙ под создаётся свой PVC
    - metadata: { name: data }
      spec: { accessModes: [ReadWriteOnce], resources: { requests: { storage: 5Gi } } }
---
kind: DaemonSet # apps/v1 · ровно ОДИН под на каждую ноду (агенты, CNI, логи)
spec:
  selector: { matchLabels: { app: node-agent } }
  template:
    spec:
      tolerations: # чтобы сесть и на control-plane (taint)
        - { key: node-role.kubernetes.io/control-plane, effect: NoSchedule }
```

LOCAL hpa.yaml + cronjob.yaml + networkpolicy.yaml

```
kind: HorizontalPodAutoscaler # autoscaling/v2 · скейлит по нагрузке
spec:
  scaleTargetRef: { kind: Deployment, name: scott } # КОГО скейлить
  minReplicas: 2
  maxReplicas: 10
  metrics: # по какой метрике (нужен metrics-server)
    - { type: Resource, resource: { name: cpu, target: { type: Utilization, averageUtilization:
---
kind: CronJob # batch/v1 · запуск Job по расписанию
spec:
  schedule: "0 3 * * *" # cron: каждый день в 03:00
  jobTemplate:
    spec: { completions: 1, parallelism: 1, /* template пода с restartPolicy: OnFailure */ }
---
kind: NetworkPolicy # networking.k8s.io/v1 · firewall (исполняет Calico)
spec:
  podSelector: {} # {} = ВСЕ поды namespace
  policyTypes: [ Ingress ] # default-deny на вход: ничего не разрешено → всё закрыто
```

Разбор ключевых полей

ПОЛЕ	ЧТО ДЕЛАЕТ / ЕСЛИ УБРАТЬ
SS · <code>serviceName</code>	headless-Service для стабильных DNS-имён <code>pg-0.pg-headless</code> . Убрать — нет предсказуемой адресации реплик.
SS · <code>volumeClaimTemplates</code>	Каждому поду свой PVC, переживает пересоздание. Заменить на общий том — реплики затрут данные друг друга.
DS · <code>tolerations</code>	Позволяет поду сесть на ноду с taint (control-plane). Убрать — DaemonSet пропустит master-ноды.
HPA · <code>scaleTargetRef / min-max</code>	Кого и в каких границах масштабировать. min=max — автоскейл выключен; нет metrics-server — метрик нет, не скейлит.
Job · <code>completions / parallelism</code>	Сколько успешных запусков нужно и сколько параллельно. completions:1 — одна успешная отработка и стоп.
CronJob · <code>schedule</code>	Cron-выражение запуска. Ошибка в синтаксисе — Job не создаётся, ошибка в Events CronJob.
NetPol · <code>podSelector {}</code>	Пустой = все поды namespace. С <code>policyTypes: [Ingress]</code> без правил = default-deny : входящий трафик закрыт, пока явно не разрешишь.

CLUSTER применяем и проверяем

```
kubectl apply -f statefulset.yaml -f hpa.yaml -f cronjob.yaml -f networkpolicy.yaml -n dev
kubectl get statefulset,pvc -n dev           # поды pg-0..2, у каждого свой PVC data-pg-N
kubectl get daemonset -n dev -o wide         # DESIRED = числу нод (по поду на ноду)
kubectl get hpa -n dev                       # TARGETS cpu%/70%, MIN/MAX, REPLICAS
kubectl get cronjob,job -n dev                # SCHEDULE, LAST SCHEDULE, статус job
kubectl get networkpolicy -n dev              # POD-SELECTOR на какие поды действует
```

GET STATEFULSET → ОЖИДАЕМО

```
NAME    READY   AGE    # pg-0, pg-1, pg-2 поднимаются ПО ОЧЕРЕДИ (0→1→2)
pg      3/3     2m     # у каждого свой PVC: data-pg-0, data-pg-1, data-pg-2
```

Читаем: StatefulSet поднимает реплики строго по порядку и даёт каждой свой PVC — поэтому он, а не Deployment, для БД. DaemonSet **DESIRED=число нод**. HPA без metrics-server покажет **<unknown>/70%** и скейлить не будет.

СОБЕС

«Deployment — для stateless. StatefulSet — стабильные имена + свой PVC на реплику (БД). DaemonSet — по поду на ноду (агенты). HPA — автоскейл по метрикам. Job/CronJob — разовые задачи. NetworkPolicy с пустым podSelector и policyTypes: [Ingress] — это default-deny, и исполняет её CNI (Calico)».



Под висит Pending — его никто не взял на ноду

Симптом: `kubectl get pods` показывает под в `Pending` минутами, контейнер даже не стартовал. Это работа **scheduler**: он не нашёл ноду, куда под влезает по правилам. Причина всегда в **Events** — туда и смотрим первым делом.

CLUSTER шаг 1 — читаем Events пода (тут вся причина)

```
kubectl -n dev get pods # STATUS = Pending, READY 0/1
kubectl -n dev describe pod web-7d9c-abcde # листаем В НИЗ до Events
```

ВЫВОД — СЕКЦИЯ EVENTS (ТО, ЧТО БЬЁТ)

```
Events:
  Type          Reason             Message
  --          -
Warning        FailedScheduling   0/3 nodes are available: 1 node(s) had intolerated
                                     taint, 2 Insufficient cpu. preemption: 0/3 nodes.
```

Как читать: `0/3 nodes are available` — scheduler проверил 3 ноды, ни одна не подошла. Хвост строки — **точная причина**: `Insufficient cpu` (мало ресурсов), `intolerated taint` (нода помечена), либо `node(s) didn't match node selector`.

CLUSTER шаг 2 — добиваем по подсказке из Events

```
kubectl get nodes # есть ли вообще Ready-ноды
kubectl describe node node3 | grep -A6 Allocatable # сколько cpu/мет свободно
kubectl describe node node3 | grep -i Taints # чем нода «отравлена»
kubectl -n dev get pvc # PVC тоже Pending? тогда дело в нём
```

ВЫВОД

```
Allocatable:  cpu: 1900m    memory: 3600Mi # потолок ноды
Taints:       node-role.kubernetes.io/control-plane:NoSchedule
web-data     Pending   ""    standard # PVC без тома → под ждёт его
```

ПРИЧИНА	КАК ПРОВЕРИТЬ	ФИКС
requests > Allocatable (мало cpu/mem)	Events: Insufficient cpu/memory ; сравни requests с Allocatable	Снизить resources.requests или добавить/освободить ноду
nodeSelector / affinity ни к чему не подходит	Events: didn't match node selector ; kubectl get nodes --show-labels	Поправить лейбл на ноде или nodeSelector в манифесте
Taint на ноде без toleration у пода	Events: untolerated taint ; describe node grep Taints	Добавить tolerations поду или снять taint с ноды
PVC в Pending (нет тома / StorageClass)	get pvc → Pending; Events PVC: no persistent volumes available	Поднять provisioner / задать существующий storageClassName

✓ Фикс по горячим следам

В 8 из 10 случаев это **ресурсы**: занизь `resources.requests.cpu` в Deployment (напр. `250m` вместо `1`) и под влезет. Если в Events `taint` — нода это control-plane с `NoSchedule`: либо разрешённый под получает `tolerations`, либо просто не планируй приложение на мастер.



ImagePullBackOff — нода не смогла скачать образ

Симптом: под в `ErrImagePull`, затем `ImagePullBackOff` (kubelet ждёт перед повтором). Под назначен на ноду, но `cri-o` не вытянул образ. У нас runtime — `cri-o`, поэтому образы на ноде смотрим через `crictl`, а не `docker`.

CLUSTER шаг 1 — за что именно ругается pull

```
kubectl -n dev describe pod api-xxxx | grep -A8 Events
```

ВЫВОД — EVENTS

```
Warning   Failed    Failed to pull image "myregistry.io/api:lattest":  
           reading manifest lattest: manifest unknown  
Warning   Failed    Error: ErrImagePull  
Normal    BackOff   Back-off pulling image "myregistry.io/api:lattest"
```

Как читать: `manifest unknown` — такого тега в registry нет (тут опечатка `lattest`). `unauthorized` / `401` — приватный registry без secret. `no such host` — нода не резолвит/не видит registry. `no matching manifest for amd64` — образ не под архитектуру ноды.

NODE3 шаг 2 — проверяем pull руками на ноде (`cri-o`)

```
crictl images | grep api           # есть ли образ на ноде вообще  
crictl pull myregistry.io/api:1.4 # повторяем pull и читаем ТОЧНУЮ ошибку
```

ВЫВОД CRICTL PULL

```
FATA[0002] pulling image: rpc error: code = Unknown  
desc = unauthorized: authentication required  # нужен imagePullSecret
```

NODE-LOCAL шаг 3 — приватный образ доставить на ноду без registry

```
# local: выгрузить образ в tar и залить на ноду  
podman save myregistry.io/api:1.4 -o api.tar  
scp -i ~/.ssh/bm_k8s api.tar root@node3:/tmp/  
  
# node3: загрузить в локальное хранилище cri-o  
ssh -i ~/.ssh/bm_k8s root@node3 'podman load -i /tmp/api.tar'  
crictl images | grep api           # теперь образ виден runtime
```

ПРИЧИНА	КАК ПРОВЕРИТЬ	ФИКС
Опечатка/нет такого тега	Events: manifest unknown ; crictl pull ту же ошибку	Исправить image: в манифесте на существующий тег
Приватный registry без secret	Events/pull: unauthorized / 401	Создать docker-registry secret + imagePullSecrets
Образа нет на этой ноде, registry недоступен	no such host / timeout; под на конкретной ноде	podman save → scp → podman load на ноде
Чужая архитектура (arm образ на amd64)	Events: no matching manifest for linux/amd64	Собрать multi-arch / тянуть образ под платформу ноды

✓ Фикс по горячим следам

Приватный registry — один секрет решает всё:

`kubectl -n dev create secret docker-registry regcred --docker-server=myregistry.io --docker-username=...`
 затем в pod-спеке `imagePullSecrets: [{name: regcred}]`. После правки тега под не перетягивает образ сам — снеси его: `kubectl -n dev delete pod api-xxxx` (Deployment поднимет новый).



CrashLoopBackOff — контейнер стартует и сразу падает

Симптом: образ скачался, под стартует, но тут же умирает — и так по кругу, `restartCount > 5`, статус `CrashLoopBackOff`. kubelet делает паузу перед перезапуском (back-off растёт до 5 мин). Главный ключ — логи прошлой попытки и Exit Code.

CLUSTER шаг 1 — логи УПАВШЕГО контейнера + код выхода

```
kubectl -n dev logs api-xxxx --previous # --previous = лог ПРОШЛОГО, упавшего
kubectl -n dev describe pod api-xxxx | grep -A6 'Last State'
```

ВЫВОД

```
panic: dial tcp 10.96.0.10:5432: connect: connection refused
Last State:      Terminated
Reason:          Error
Exit Code:       1 # приложение само упало (не OOM/не сигнал)
Restart Count: 7
```

Как читать: `Exit Code` сразу делит причины. `1` — упало само приложение (смотри текст лога). `137` — убито по памяти (`OOMKilled`). `143` — получило SIGTERM (часто liveness-probe прибывает живой под). `0` — процесс просто завершился, а под не Job (кривой ENTRYPOINT/CMD).

EXIT CODE → ПРИЧИНА	КАК ПРОВЕРИТЬ	ФИКС
<code>1</code> — приложение падает (паника/конфиг/нет зависимости)	<code>logs --previous</code> — там стектрейс/ошибка	Чинить код/ENV/конфиг; поднять зависимость (БД и т.п.)
<code>137</code> — <code>OOMKilled</code> (память > limit)	describe: <code>Reason: OOMKilled</code> ; <code>kubectl top pod</code>	Поднять <code>resources.limits.memory</code> или починить утечку
<code>143</code> — SIGTERM, liveness бьёт ЗДОРОВЫЙ под	Events: <code>Liveness probe failed</code> у работающего сервиса	Поправить путь/порт/таймауты probe или добавить <code>startupProbe</code>
<code>0</code> — процесс вышел, но это не Job	<code>logs</code> чисто, контейнер «отработал и закрылся»	Дать долгоживущий процесс в <code>command/ENTRYPOINT</code>

⚠ Не путай: упало приложение vs его убил k8s

`Exit Code 1` + текст в логе = виноват сам контейнер, k8s ни при чём. `137 OOMKilled` и `143` = под был здоров, его **прибил кластер** (память/probe). Это разные ремонты: в первом случае лезешь в код, во втором — в манифест (limits/probes). Сначала Exit Code, потом гипотеза.

✓ Фикс по горячим следам

Образ есть, а лог пустой и под падает мгновенно — почти всегда кривой старт. Перебей команду на «живую» для отладки: `kubectl -n dev run dbg --rm -it --image=твой-образ --command -- sh` и запусти бинарь руками — увидишь реальную ошибку без back-off. Если `143` от liveness — временно убери probe, убедись что под живёт, потом чини порт/путь probe.



Running, но 0/1 Ready — трафик в под не идёт

Симптом: под `Running`, не рестартует, но `READY` = `0/1`. Контейнер жив, но `readiness probe` не зелёная — значит k8s считает под «ещё не готов обслуживать» и **выкидывает его из endpoints Service**. Снаружи это выглядит как «сервис не отвечает / 503».

CLUSTER шаг 1 — какое условие красное и почему probe падает

```
kubectl -n dev get pods # READY 0/1, но STATUS Running
kubectl -n dev describe pod web-xxxx | grep -A6 Conditions
kubectl -n dev describe pod web-xxxx | grep -i 'Readiness probe'
```

Вывод

Conditions:

```
Type          Status
Ready          False      # ← под НЕ ГОТОВ
ContainersReady False
Warning Unhealthy Readiness probe failed: HTTP probe failed
with statuscode: 404 # probe стучит не туда
```

Как читать: `Ready False` + `Readiness probe failed` — приложение работает, но по адресу probe отвечает не так, как ждёт k8s. `statuscode 404` — неверный `path`. `connection refused` — неверный `port` или сервис ещё поднимается. `timeout` — приложение медленно стартует.

CLUSTER шаг 2 — доказываем, что под выпал из endpoints

```
kubectl -n dev get endpointslices -l kubernetes.io/service-name=web
# дёргаем сам endpoint probe ИЗНУТРИ пода — что реально отвечает
kubectl -n dev exec web-xxxx -- wget -qO- localhost:8080/healthz
```

Вывод

```
ADDRESSES  READY
<none>     # адресов нет → Service пустой → 503 снаружи
wget: server returned error: HTTP/1.1 404 Not Found # /healthz нет
```

ПРИЧИНА	КАК ПРОВЕРИТЬ	ФИКС
Неверный <code>path</code> readiness probe	probe failed <code>statuscode 404</code> ; <code>exec ... wget /healthz</code>	Указать реальный путь (<code>/health</code> , <code>/</code>) в <code>readinessProbe.httpGet.path</code>
Неверный <code>port</code> probe	probe: <code>connection refused</code> ; сверь с <code>containerPort</code>	Выставить порт приложения в <code>readinessProbe.httpGet.port</code>
Приложение медленно стартует (probe рано бьёт)	сначала Unhealthy, потом сам зеленеет; долгий старт в логах	Добавить <code>startupProbe</code> или поднять <code>initialDelaySeconds</code>
Зависимость не готова (БД/кэш), приложение держит «not ready»	лог: ждёт коннекта; <code>/healthz</code> отдаёт 503 осознанно	Поднять зависимость; это штатная защита — под верно вне трафика

i readiness ≠ liveness

readiness провалилась → под убирают из `endpoints`, но НЕ перезапускают (ждут, пока починится сам).

liveness провалилась → под **перезапускают**. Поэтому «Running 0/1 навсегда» — это всегда **readiness**: контейнер жив, но в ротацию его не пускают.

✓ Фикс по горячим следам

Проверь probe не из манифеста, а изнутри:

```
kubectl -n dev exec web-xxxx -- curl -s -o /dev/null -w "%{http_code}" localhost:ПОРТ/ПУТЬ.
```

Получил `200` — значит в probe просто не тот путь/порт, правь `readinessProbe`. Медленный старт лечится `startupProbe` (он «держит» liveness/readiness, пока приложение не встанет) — это правильное, чем раздувать `initialDelaySeconds`.



Service не отвечает — а поды-то живые

Симптом: поды `Running 1/1`, а обращение к Service виснет/`connection refused`. Service сам трафик не носит — он лишь список адресов (**endpoints**), куда `kube-proxy` раскидывает пакеты. Пустой список = некуда слать. Топ-причина — **selector сервиса ≠ labels подов**.

CLUSTER шаг 1 — есть ли у сервиса живые endpoints (ГЛАВНОЕ)

```
kubectl -n dev get endpointslices -l kubernetes.io/service-name=web
kubectl -n dev get pods -l app=web -o wide # а поды под этот selector есть?
```

ВЫВОД

NAME	ADDRESSTYPE	ENDPOINTS	PORTS
web-abcd	IPv4	<unset>	<unset> # endpoints пусто!
No resources found		# под selector app=web никто не попал	

Как читать: **ENDPOINTS** пусто (или `<none>`) = Service ни на кого не указывает, любой запрос упрётся в стену. Дальше сверяем **в лоб**: какие labels стоят на подах против того, что написано в **selector** сервиса — расхождение хоть в одной паре ключ=значение рвёт связь.

CLUSTER шаг 2 — сверяем selector сервиса с labels подов + targetPort

```
kubectl -n dev get svc web -o jsonpath='{.spec.selector}{"\n"}'
kubectl -n dev get pods --show-labels | grep web
kubectl -n dev get svc web -o wide # TARGETPORT = порт ВНУТРИ пода?
```

ВЫВОД

{\"app\":\"web\"}		# selector сервиса	
web-7d9c-x	1/1	Running	app=webapp # а на поде app=webapp ≠ web!
PORT(S)	TARGETPORT		
80/TCP	8080 # бьёт в 8080, а nginx слушает 80?		

NETSHOOT шаг 3 — стучимся в сервис изнутри кластера

```
kubectl -n dev run netshoot --rm -it --image=nicolaka/netshoot --restart=Never -- bash
# внутри netshoot — проверяем цепочку имя → ClusterIP → порт:
nc -vz web 80 # открыт ли порт сервиса
curl -sv http://web:80 # отвечает ли приложение через svc
```

ВЫВОД ПРИ ПУСТЫХ ENDPOINTS

```
web (10.233.12.7:80) open # ClusterIP есть, но...
curl: (7) Failed to connect ... Connection refused # слать некуда
```

ПРИЧИНА	КАК ПРОВЕРИТЬ	ФИКС
selector ≠ labels (топ-причина)	endpoints пусто; svc selector ≠ pod labels	Привести в соответствие: spec.selector = labels подов
targetPort не тот порт в поде	endpoints есть, но TARGETPORT ≠ containerPort	Выставить targetPort = порт, который слушает приложение
Поды есть, но 0/1 Ready	endpoints пусто, поды не Ready (см. плейбук 4)	Починить readiness — под вернётся в endpoints
Не тот namespace в имени	из netshoot web не резолвится, а web.dev да	Обращаться web.dev.svc.cluster.local или из того же ns

✓ Фикс по горячим следам

Цепочка проста: **Service** → **endpoints** → **Pod**. Пустой endpoints почти всегда = **selector сервиса не совпал с labels подов**. Сверь две строки (`get svc -o jsonpath='{.spec.selector}'` vs `get pods --show-labels`) и выровняй. Endpoints наполнились, а всё равно молчит — тогда это **targetPort** : он должен указывать на порт **внутри пода**, а не на порт сервиса.



DNS не резолвится — по имени не ходит, по IP ходит

Симптом: из пода `curl http://web` даёт `could not resolve host`, а по `ClusterIP` напрямую работает. Значит сеть жива, лёг **DNS**. В кластере имена резолвит **CoreDNS** (поды в `kube-system`), а поды ходят к нему по адресу из своего `/etc/resolv.conf`.

NETSHOOT шаг 1 — резолвим по полному FQDN и смотрим `resolv.conf`

```
kubectl -n dev run netshoot --rm -it --image=nicolaka/netshoot --restart=Never -- bash
# внутри netshoot:
cat /etc/resolv.conf                                # кто наш DNS и какой search-домен
nslookup web.dev.svc.cluster.local                  # резолвим ПОЛНЫМ именем
nslookup kubernetes.default                          # базовый сервис — резолвится всегда

ВЫВОД КОГДА COREDNS НЕДОСТУПЕН
nameserver 10.233.0.10                               # ClusterIP CoreDNS (kube-dns)
search dev.svc.cluster.local svc.cluster.local cluster.local
;; connection timed out; no servers could be reached # DNS не отвечает
```

Как читать: `no servers could be reached` — пакет до CoreDNS не дошёл (поды лежат или `NetworkPolicy` режет :53). `NXDOMAIN` на коротком имени, но **успех** на FQDN — кривой `search`-домен или обращаешься в чужой namespace. `SERVFAIL` — CoreDNS жив, но сам не может разрезолвить (внешний апстрим).

CLUSTER шаг 2 — живы ли поды CoreDNS и его сервис

```
kubectl -n kube-system get pods -l k8s-app=kube-dns -o wide # Running? Restarts?
kubectl -n kube-system get svc kube-dns                     # ClusterIP = 10.233.0.10
kubectl -n kube-system logs -l k8s-app=kube-dns --tail=30 # ошибки CoreDNS

ВЫВОД — COREDNS В CRASHLOOP
coredns-xxx 0/1 CrashLoopBackOff 8 # DNS лёг → резолва нет ни у кого
coredns-yyy 1/1 Running          0
kube-dns ClusterIP 10.233.0.10 <none> 53/UDP,53/TCP
```

ПРИЧИНА	КАК ПРОВЕРИТЬ	ФИКС
Поды CoreDNS лежат (CrashLoop/0 реплик)	<code>get pods -n kube-system -l k8s-app=kube-dns</code> ; логи	Перезапустить: <code>rollout restart deploy/coredns -n kube-system</code>
Неверный FQDN / чужой namespace	FQDN резолвится, короткое имя — <code>NXDOMAIN</code>	Звать <code>svc.ns.svc.cluster.local</code> или из того же ns
NetworkPolicy режет порт :53 к CoreDNS	<code>no servers reached</code> ; есть NP в ns; по IP CoreDNS не пускает	Разрешить egress udp/tcp 53 в <code>kube-system</code>
CoreDNS не резолвит ВНЕШНИЕ имена (апстрим)	внутрикластерные имена ок, <code>google.com</code> — <code>SERVFAIL</code>	Поправить <code>forward</code> в Corefile (ConfigMap <code>coredns</code>)

✓ Фикс по горячим следам

Разделяй «лёг DNS» и «не то имя»: если по `ClusterIP` ходит, а по имени нет — виноват резолв, не сеть. CoreDNS в CrashLoop — подними `kubectl -n kube-system rollout restart deployment coredns`. Резолв есть, но только по полному FQDN — это `search` -домен: внутри своего ns зови коротко (`web`), в чужой ns — `web.dev` или полным `web.dev.svc.cluster.local`.



Нода NotReady и сеть между нодами — Calico/VXLAN

Симптом А: `kubectl get nodes` показывает ноду `NotReady`, поды на ней зависают в `Terminating` /переезжают. **Симптом Б:** поды на **одной** ноде друг друга видят, а на **разных** — нет (`curl` между нодами виснет). Б — почти всегда **оверлей Calico/VXLAN**: пакет не доезжает по физсети.

CLUSTER А · шаг 1 — почему нода NotReady (Conditions)

```
kubectl get nodes # кто NotReady
kubectl describe node node3 | grep -A8 Conditions # что зажглось
```

ВЫВОД — CONDITIONS НОДЫ

```
node3   NotReady   <none>   v1.3x.x
MemoryPressure   True    # кончается память
DiskPressure     False
Ready            False   KubeletNotReady # kubelet не репортит health
```

Как читать: `MemoryPressure/DiskPressure True` — на ноде кончились ресурсы, kubelet начнёт вытеснять поды.
`Ready False · KubeletNotReady` — kubelet не достукивается до api-server либо упал.
`runtime network not ready` — лёг CNI (Calico) на ноде. Дальше идём на ноду по SSH и читаем демоны.

NODE3 А · шаг 2 — на ноде: kubelet и cri-o

```
journalctl -u kubelet -xe | tail -40 # на что ругается kubelet
journalctl -u crio -xe | tail -40    # жив ли runtime (cri-o)
df -h / ; free -m                   # реально ли кончились диск/память
systemctl restart kubelet           # после фикса ресурсов
```

ВЫВОД JOURNALCTL KUBELET

```
"Failed to start cni network" err="no CNI configuration" # Calico не встал
node3 status is now: NodeNotReady
```

NODE1·NODE3 Б · cross-node: оверлей и UDP 4789 (VXLAN)

```
# все calico-node должны быть Running на BCEX нодах:
kubectl -n kube-system get pods -l k8s-app=calico-node -o wide

# на ноде — есть ли интерфейс оверлея и пул:
ip -d link show vxlan.calico # интерфейс VXLAN поднят?
ip route | grep 10.233       # маршруты на pod-сети ДРУГИХ нод
ss -ltnp | grep 4789         # VXLAN слушает UDP 4789?
```

ВЫВОД КОГДА МЕЖНОВОДОВАЯ СЕТЬ БИТА

```
calico-node-xxx 0/1 Running # не Ready → BGP/VXLAN не сошёлся
Device "vxlan.calico" does not exist # оверлей-интерфейса нет
# (UDP 4789 закрыт фаерволом между нодами — пакеты дропаются)
```

ПРИЧИНА	КАК ПРОВЕРИТЬ	ФИКС
Memory/DiskPressure на ноде	describe node Conditions True ; df -h , free -m	Освободить диск/память, затем systemctl restart kubelet
kubelet/cri-o лёг или не видит api-server	journalctl -u kubelet/-u cric ; systemctl status	Перезапустить демон; проверить токен/сертификаты, время (NTP)
Заблокирован UDP 4789 между нодами (VXLAN)	cross-node curl виснет; ss -ltnp grep 4789 ; фаервол/SG	Открыть UDP 4789 между всеми нодами по приватной сети
calico-node не Ready / нет vxlan.calico	get pods -l k8s-app=calico-node 0/1; нет интерфейса	rollout restart ds/calico-node -n kube-system ; сверить ipools

✓ Фикс по горячим следам

Делишь по симптому: **NotReady** → лезешь на ноду в `journalctl -u kubelet` и смотришь ресурсы (`df -h` / `free -m`). Поды на одной ноде видят, на разных нет → это оверлей: проверь, что все `calico-node` Ready, поднят `vxlan.calico`, и между нодами открыт `UDP 4789` (классика на облачных VM — забытый firewall/security-group режет VXLAN, и cross-node трафик тихо дропается).



Контейнеры и Docker: вопросы с подвохом

Вопросы, которые реально звучат на интервью по Блоку 1. У каждого — **ловушка** (на чём валят), **сильный ответ** и, где уместно, формулировка **по грейдам** (jr / md / sr).

1 Чем контейнер отличается от виртуальной машины?

ЛОВУШКА

«контейнер — лёгкая виртуалка». У VM своё ядро, у контейнера ядро общее с хостом.

ОТВЕТ

VM поднимает полную гостевую ОС со своим ядром. Контейнер — изолированный **процесс** поверх ядра хоста: легче, быстрее, меньше ресурсов. Честный минус: у VM изоляция жёстче.

JR / MD / SR

jr: «легче, делит ядро» · md: namespaces+cgroups · sr: общий kernel = общая поверхность атаки, gVisor/Kata.

2 За счёт чего достигается изоляция контейнера?

ЛОВУШКА

свалить всё в «namespaces». Изоляция и лимиты — разные механизмы.

ОТВЕТ

namespaces — что процесс видит (PID, сеть, mount, hostname); **cgroups** — сколько ресурсов съест (CPU/RAM). Ядро общее. Контейнер = процесс хоста со своими пространствами имён и лимитами.

3 Чем слой отличается от стейджа и зачем multistage?

ЛОВУШКА

путать слой и стейдж — спрашивают прицельно.

ОТВЕТ

Слой = инструкция `RUN/COPY/ADD` (кэш). **Стейдж** = блок `FROM`. **Multistage**: собрал бинарь со всем тулчейном, в маленький рантайм скопировал только его. Naive ~302 MB vs multistage+Alpine ~18 MB (×17), код один.

JR / MD / SR

md: `go.mod` + `go mod download` кладут ДО `COPY . .` — кэш зависимостей цел.

4 `docker build` vs `docker run` ?

ЛОВУШКА

«run сам соберёт, если образа нет». Нет — run только тянет готовый образ.

ОТВЕТ

build = из Dockerfile собрать **ОБРАЗ**. **run** = из образа запустить **КОНТЕЙНЕР** (run = create + start). Образ — «класс», контейнер — «экземпляр».

5 Alpine или distroless? Как дебажить distroless?

ЛОВУШКА

«distroless всегда лучше». Меньше и безопаснее, но shell нет — `exec -- sh` не зайдёт.

ОТВЕТ

Alpine — когда надо зайти внутрь (есть `sh`). **distroless** — минимум пакетов, без shell. Дебар distroless — **ephemeral-контейнером**: `kubectl debug` подсаживает временный контейнер с инструментами.



Архитектура: кто за что отвечает

6 Под какую арх образ? Что за `exec format error` ?

ЛОВУШКА

«образ универсальный». Он собирается под **платформу = ОС + арх CPU** и должен совпасть с нодой.

ОТВЕТ

ОС — Linux. Арх: **amd64** (серверы) vs **arm64** (Apple Silicon, Graviton). Собрал на Mac (arm64), ноды amd64 → `exec format error`. Лечат: `--platform linux/amd64`, cross-compile, multi-arch `buildx` — нода тянет свой вариант.

7 Из чего control plane и чем отличается от worker?

ЛОВУШКА

смешать роли — на worker нет api-server/scheduler/etcd.

ОТВЕТ

Control plane: api-server (вход), etcd (память), scheduler (нода поду), controller-manager (reconcile). **Worker:** kubelet (агент), cri-o (runtime), kube-proxy (сетевые правила), CNI (сеть подов).

JR / MD / SR

jr: «CP решает, worker выполняет» · sr: CP тоже может нести поды (taint снят).

8 Кто пишет в etcd?

ЛОВУШКА

«scheduler и kubelet пишут напрямую». Никто, кроме api-server.

ОТВЕТ

В etcd пишет **ТОЛЬКО api-server**. Все (scheduler, controller-manager, kubelet) watch'ат api-server и пишут результат обратно через него. etcd — единая точка правды за api-server.

9 kube-proxy — это прокси, через который идёт трафик?

ЛОВУШКА

само название. Трафик через kube-proxy НЕ идёт.

ОТВЕТ

kube-proxy **заранее пишет сетевые правила** (`iptables` / `IPVS`) на каждой ноде, чтобы трафик на ClusterIP попадал в поды. Пакеты идут по правилам в ядре; сам процесс в datapath не участвует.

JR / MD / SR

md: «пишет DNAT Service→PodIP» · sr: iptables vs IPVS (хэш на тысячах svc), источник — EndpointSlice.

10 Что такое CRI, CNI, CSI?

ЛОВУШКА

назвать продукт вместо интерфейса. cri-o/Calico/Сeph — реализации.

ОТВЕТ

Контракты kubelet к подсистемам: **CRI** — runtime (cri-o, containerd); **CNI** — сеть подов (Calico, Cilium); **CSI** — хранилище (Сeph, EBS). K8s задаёт интерфейс, вендор — реализацию.



etcd, кворум, сеть и типы Service

11 Что такое кворум и зачем etcd нечётное число нод?

ЛОВУШКА

«больше нод = надёжнее, ставь 6». 6 терпит столько же отказов, что и 5.

ОТВЕТ

Кворум = большинство нод, согласных принять запись: $N/2 + 1$ ($3 \rightarrow 2, 5 \rightarrow 3$). etcd на **Raft**: лидер, запись коммитится при подтверждении большинством (sr: leader election, term, commit-index). Число **нечётное**: 5 и 6 терпят по 2 отказа, шестая лишь повышает риск split-brain.

12 5 нод etcd, 3 упали. Что с кластером?

ЛОВУШКА

«всё умерло». Нет: нагрузка живёт, отказывает только запись.

ОТВЕТ

5 нод $\rightarrow$ кворум 3. Осталось $2 < 3 \rightarrow$ лидера не выбрать, записи не коммитятся $\rightarrow$ кластер **read-only**. Поды работают, трафик идёт (kubelet автономен). Чинить: вернуть ноды или `etcdctl snapshot restore` из бэкапа.

13 На каком порту работает `ping` ?

ЛОВУШКА

назвать любой порт. У ping портов НЕТ.

ОТВЕТ

Ни на каком. ping — это **ICMP** (L3: есть адреса, портов нет). Порты только на **L4** (TCP/UDP). Вывод: **ping прошёл $\neq$ сервис работает** — хост лишь ответил на ICMP. Порт проверяй `nc -vz host 443`; ICMP может быть закрыт firewall'ом.

14 Как пакет идёт от пода к поду на РАЗНЫХ нодах (Calico VXLAN)?

ЛОВУШКА

«поды в одной плоской сети». Между нодами пакет инкапсулируется.

ОТВЕТ

У пода `eth0` — конец veth-пары, второй на ноде = `caliXXXX`. Для удалённого пода маршрут ведёт в `vxlan.calico`: пакет **инкапсулируется в VXLAN/UDP**, идёт по физсети, там распаковывается. Режим проверяю `get ippools -o yaml | egrep vxlanMode`.

JR / MD / SR

md: veth + vxlan.calico · sr: VXLAN / IPIP(`tunl0`) / native-BGP (реальный pod-IP).

15 Чем отличаются ClusterIP, NodePort, LoadBalancer?

ЛОВУШКА

«LoadBalancer работает везде». On-prem без MetalLB он висит в `<pending>`.

ОТВЕТ

ClusterIP — только внутри, по DNS (дефолт). **NodePort** — один порт на КАЖДОЙ ноде наружу. **LoadBalancer** — внешний IP от облака/MetalLB; у нас `<pending>`. Вложены: LB поверх NodePort поверх ClusterIP.

JR / MD / SR

sr: + Headless (DNS отдаёт IP подов, StatefulSet) и ExternalName (CNAME наружу).



Service, DNS и виды контроллеров

16 Как Service находит поды? Почему endpoints пустые?

ЛОВУШКА

«знает поды по имени». Только по **labels/selector**; сломал selector → endpoints пусты.

ОТВЕТ

Service ищет поды по **labels** через **selector**, не по имени. Живые backend'ы (**PodIP:port**) лежат в **EndpointSlice**. Selector не совпал → endpoints пусты, **curl** висит. Дебаг: **kubectl get endpoints <svc>**.

JR / MD / SR

md: сразу **get endpoints** · sr: причины — опечатка labels, readiness не прошёл, не тот порт.

17 **externalTrafficPolicy** : Local vs Cluster?

ЛОВУШКА

не знать про потерю source IP. При Cluster реальный IP теряется (SNAT).

ОТВЕТ

Cluster (дефолт): балансит на под на ЛЮБОЙ ноде — лишний хоп, source IP теряется (SNAT). **Local**: только на под на ТОЙ ЖЕ ноде; нет пода → дроп; зато source IP **сохраняется**, нет хопа. Local — когда нужен реальный IP (логи/geo).

JR / MD / SR

sr: **internalTrafficPolicy** (то же внутри), Local + DaemonSet чтобы не дропать.

18 Как под находит сервис по имени? Что такое FQDN, CoreDNS?

ЛОВУШКА

«куб лезет в **/etc/hosts**». Имена резолвит **CoreDNS**.

ОТВЕТ

DNS в кластере держит **CoreDNS**. Короткое **web** в своём ns → ClusterIP; **FQDN**: **web.<ns>.svc.cluster.local**. Кросс-ns — через ns в имени (**web.prod**). Проверка: **dig web.net-demo.svc.cluster.local +short**.

JR / MD / SR

sr: **search** -домены в **/etc/resolv.conf** — поэтому короткое имя только в своём ns.

19 Deployment vs StatefulSet vs DaemonSet — когда что?

ЛОВУШКА

«StatefulSet для баз данных». Правильно — «для **stateful**»; БД лишь пример.

ОТВЕТ

Deployment — stateless: rolling update, реплики взаимозаменяемы. **StatefulSet** — stateful: стабильные имена (**web-0/1/2**), свой PVC на под, порядок 0→N. **DaemonSet** — по поду на КАЖДОЙ ноде (лог-агенты, node-exporter).

JR / MD / SR

sr: headless-svc + volumeClaimTemplates у STS, tolerations у DS на control-plane.



Сущности: обновление, пробы, ресурсы

20 Что при rolling update? Откуда два ReplicaSet?

ЛОВУШКА

«обновляет поды на месте». Нет — создаёт НОВЫЙ RS, старый в 0.

ОТВЕТ

Создаётся **новый RS** (растёт 0→N), старый падает N→0 — по шагу **maxSurge** (сверх replicas) и **maxUnavailable** (потерять в моменте). Старый RS в истории → `rollout undo`. Альтернатива — `Recreate` (с даунтаймом).

JR / MD / SR

sr: в проде откат через Helm/CI, а не `kubectl rollout undo`.

21 Чем PDB отличается от персистентности и от maxUnavailable?

ЛОВУШКА

«PDB бережёт данные». Нет — про ДОСТУПНОСТЬ подов; данные — PV/PVC.

ОТВЕТ

PDB ограничивает, сколько подов выключено разом при **добровольных** disruptions (`drain`, апдейт ноды) — `minAvailable` / `maxUnavailable` по selector. НЕ спасает от краша/OOM. «Файл переживает под» — это **PV/PVC**. `maxUnavailable` в Deployment рулит rolling update, не drain'ом.

22 Три probe: liveness, readiness, startup?

ЛОВУШКА

путать liveness и readiness. Одна рестартит, другая выводит из трафика.

ОТВЕТ

liveness — жив ли процесс; провал → kubelet **рестартит**. **readiness** — готов ли к трафику; провал → под **выпадает из endpoints** (не рестартится). **startup** — для медленного старта: пока идёт, liveness/readiness не считаются.

23 `requests` vs `limits` : scheduler и OOMKilled?

ЛОВУШКА

«scheduler смотрит на limits». Нет — на **requests**.

ОТВЕТ

requests — гарантия: по ним **scheduler** выбирает ноду. **limits** — потолок: превысил memory-limit → `OOMKilled`; превысил CPU-limit → **тротлинг** (не убивают). Нет ноды под requests → под в `Pending`.

JR / MD / SR

sr: QoS-классы (Guaranteed/Burstable/BestEffort), порядок eviction под памятью.

24 ConfigMap/Secret: env vs volume — разница на практике?

ЛОВУШКА

«поменял ConfigMap — под подхватит». Через env — НЕТ, нужен рестарт.

ОТВЕТ

Подключают как **env** ИЛИ **файлы через volume**. Через env запущенный под изменение не подхватит — нужен рестарт Pod/Deployment. Volume-файлы kube обновит на лету, но приложение должно само перечитать.



Хранилище, RBAC и вся цепочка apply

25 PVC висит в `Pending` . Почему на on-prem?

ЛОВУШКА

«в облаке работало». На голом kubespray HET default StorageClass и provisioner'a.

ОТВЕТ

Без **default StorageClass** и provisioner'a PVC висит в `Pending` навсегда (и STS с БД не поднимется). В облаке есть CSI-драйвер и default-SC. On-prem ставишь сам: `local-path` (dev), NFS (RWX), Ceph/Rook/Longhorn (прод).

JR / MD / SR

sr: accessModes RWO/RWX, reclaimPolicy Retain/Delete, volumeClaimTemplates.

26 RBAC: Role vs ClusterRole. Зачем ServiceAccount?

ЛОВУШКА

путать область. Role — в namespace, ClusterRole — весь кластер.

ОТВЕТ

Role/RoleBinding — один namespace; **ClusterRole/CRB** — весь кластер (узлы, PV). **ServiceAccount** — «юзер» для подов внутри кластера: поду выдают SA, чтобы он ходил в api-server с ограниченными правами, не под админом.

27 Порядок в api-server: authn / authz / admission? Secret шифрован?

ЛОВУШКА

перепутать порядок; «Secret — это шифрование». **base64 ≠ шифрование**.

ОТВЕТ

Строго: **authn** (кто — серт/токен) → **authz** (что можно — RBAC) → **admission** (изменит/запретит) → валидация → etcd. Secret в etcd по умолчанию НЕ зашифрован (base64, виден открытым) — защита: **encryption-at-rest** + RBAC + Vault, монтаж в tmpfs.

28 Опиши всю цепочку `kubectl apply` . Когда self-healing НЕ работает?

ЛОВУШКА

«создался под». Правильно — ВСЯ reconcile-цепочка; голый Pod не самовосстанавливается.

ОТВЕТ

kubectl → **api-server** (authn→authz→admission) → **etcd** → **controller-manager** (Deployment→RS→Pod-объекты) → **scheduler** (filter→score→bind, `nodeName`) → **kubelet** (CRI/CNI/CSI) → `Running/Ready`. Self-healing даёт только контроллер; голый **Pod** удалили — не вернётся. **Pending** = scheduler не нашёл ноду (мало requests, taints, nodeSelector, нет provisioner'a).

JR / MD / SR

md: этапы по компонентам · sr: watch + level-triggered reconcile — та же машина запускает и самовосстанавливает.

i Как отвечать на собеседе

Сначала минимальный корректный ответ → «рассказать подробнее?» → углубляй по грейду. Не топи интервьюера сразу etcd/CNI/RBAC, но держи запас на «а если копнуть?». kubeconfig для прода — отдельный SA, не `admin.conf`.



8 заданий — каждое с промтом для ИИ

Тетрадь работает, только если повторяешь руками. Подними свой kubespray-кластер (раздел выше) и пройди 8 заданий. К каждому — промт для нейронки: проси **объяснить** → **дать команды** для проверки → **задать тебе вопрос**.

№1 · Docker: плохой vs хороший образ

Собери `Dockerfile.simple` и multistage+Alpine, сравни размеры (должно быть ×15), зайди внутрь Alpine через `sh`.



ПРОМТ

Вот мой Dockerfile [вставь]. Объясни, почему образ большой, перепиши через multistage + Alpine (чтобы можно зайти внутрь sh), собери, сравни размеры и поясни каждую строку.

№2 · Контейнер = namespaces + cgroups

Через `/proc/PID/ns`, `nsenter`, `cgroup` докажи, что контейнер — процесс хоста с изоляцией и лимитами.



ПРОМТ

Я запустил контейнер на Linux. Дай команды показать его namespaces (через `/proc` и `nsenter`) и cgroups, и объясни, почему контейнер — это процесс, а не отдельная ОС.

№3 · Docker-сеть: DNS по имени + route

Подними app+postgres+netshoot в одной сети, докажи DNS по имени сервиса и путь пакета (до postgres напрямую vs наружу через gateway).



ПРОМТ

Подними docker-compose: app(netshoot)+postgres в одной bridge-сети. Дай команды показать DNS по имени (127.0.0.11), `ip route get` до postgres vs 8.8.8.8, `veth/bridge` на хосте. Объясни путь пакета пошагово.

№4 · Раскатай свой kubespray-кластер

3 Ubuntu-VM → ansible → `get nodes` = 3 Ready (2 control-plane + 1 worker, cri-o, Calico).



ПРОМТ

Хочу поднять k8s через kubespray на 3 Ubuntu-VM (2 control-plane, 1 worker, cri-o, Calico). Проведи по шагам: `inventory`, `group_vars`, `cluster.yml`, как проверить, что кластер живой. Если упадёт — помоги разобрать ошибку из лога.



Задания 5–8 · сущности и сеть

№5 · Deployment+Service, сломай selector

Создай Deployment + Service, проверь трафик. Сломай label → увидь пустой EndpointSlice → почини. Топовый троблшутинг.

**ПРОМТ**

Создай Deployment + Service. Покажи, как Service находит поды по labels/selector. Сломай selector, объясни, почему endpoints опустели и трафик не идёт, и помоги починить.

№6 · Secret в etcd (base64) + ограниченный SA

Докажи, что Secret — base64, не шифрование. Создай SA+Role+RoleBinding (только чтение подов в dev), проверь права.

**ПРОМТ**

Создай Secret, докажи, что это base64, а не шифрование (декодни). Потом создай ServiceAccount+Role+RoleBinding, дающий ТОЛЬКО чтение подов в namespace dev, и проверь через `kubectl auth can-i --as=...`

№7 · Cross-node ping через Calico/VXLAN

2 пода на разных нодах → найди veth↔cali → пропингуй под на другой ноде, объясни путь через `vxlan.calico`.

**ПРОМТ**

На моём kubespray-кластере (Calico VXLAN) подними 2 пода netshoot на РАЗНЫХ нодах. Дай команды: показать eth0/veth внутри пода, найти cali-интерфейс на ноде, пропинговать под на другой ноде и объяснить путь через `vxlan.calico`.

№8 · Rolling update + rollback

Обнови образ → увидь, как новый ReplicaSet растёт, старый уходит в 0 → откати.

**ПРОМТ**

Покажи rolling update Deployment'a: обнови образ, дай команды увидеть, как новый ReplicaSet растёт, а старый уходит в 0. Потом откати через `rollout undo` и объясни `maxUnavailable/maxSurge`.

✓ Прошёл все 8 — ты не «зазубрил k8s», а пощупал руками

Это и отличает на собеседовании: не пересказ определений, а «я поднимал, ломал, чинил».



Состояние кластера и управление

Смотрим состояние

CLUSTER get / describe / logs / events

```
kubectl get nodes -o wide
kubectl get pods -A -o wide           # все поды всех namespace
kubectl get deploy,rs,svc,ingress -n dev
kubectl describe pod <pod> -n dev     # Events внизу — первое место для дебага
kubectl logs -f <pod> -n dev [-c <container>]
kubectl logs --previous <pod> -n dev  # логи упавшего контейнера
kubectl get events -n dev --sort-by=.lastTimestamp
```

Применяем, катим, откатываем

CLUSTER apply / scale / rollout

```
kubectl apply -f app.yaml -n dev
kubectl diff -f app.yaml -n dev       # что изменится ДО применения
kubectl scale deploy/web --replicas=5 -n dev
kubectl set image deploy/web web=nginx:1.27 -n dev
kubectl rollout status deploy/web -n dev
kubectl rollout undo deploy/web -n dev # откат на прошлую ревизию
kubectl rollout history deploy/web -n dev
```

Контекст и права

CLUSTER config / auth

```
kubectl config get-contexts
kubectl config use-context <ctx>
kubectl auth can-i create pods -n dev
kubectl auth can-i list pods --as=system:serviceaccount:dev:dev-reader -n dev
```




Сеть, конфиги, отладка пода

Сеть и сервисы

CLUSTER svc / endpoints / DNS / netshoot

```
kubectl get svc,endpointslice -n dev
kubectl get endpointslice -n dev -l kubernetes.io/service-name=web # живые backend'ы
kubectl port-forward svc/web 8080:80 -n dev # проброс на локалку
kubectl run net --rm -it --image=nicolaka/netshoot -n dev -- bash
# внутри netshoot:
curl -I web # по имени сервиса
dig web.dev.svc.cluster.local +short # ClusterIP
nc -vz web 80 # порт открыт? (ping не покажет!)
```

Конфиги и секреты

CLUSTER configmap / secret

```
kubectl create configmap app-cfg --from-literal=MODE=prod -n dev
kubectl create secret generic db --from-literal=PASSWORD=s3cr3t -n dev
kubectl get secret db -n dev -o jsonpath='{.data.PASSWORD}' | base64 -d # base64 ≠ шифрование
```

Отладка пода

CLUSTER exec / debug / cp

```
kubectl exec -it <pod> -n dev -- sh
kubectl debug -it <pod> -n dev --image=nicolaka/netshoot --target=<container> # distroless
kubectl cp dev/<pod>:/var/log/app.log ./app.log
kubectl get pod <pod> -n dev -o yaml # полный манифест как есть в кластере
```



Нода, runtime, etcd, kubespray

На ноде — runtime cri-o

NODE crictl / journalctl

```
crictl ps                # контейнеры (вместо docker ps)
crictl images
crictl logs <container-id>
crictl inspect <container-id> | grep -i pid
journalctl -u kubelet -xe | tail -40
journalctl -u crio -xe | tail -40
```

etcd — память кластера

NODE1 etcdctl (серты из ps -efww | grep etcd)

```
export ETCDCTL_API=3
E="--cacert=/etc/ssl/etcd/ssl/ca.pem --cert=/etc/ssl/etcd/ssl/node-node1.pem --key=/etc/ssl/etcd/ssl/node-node1.key"
etcdctl $E --endpoints=https://127.0.0.1:2379 endpoint health
etcdctl $E --endpoints=https://127.0.0.1:2379 endpoint status -w table # кто LEADER
etcdctl $E --endpoints=https://127.0.0.1:2379 member list
etcdctl $E --endpoints=https://127.0.0.1:2379 snapshot save backup.db # бэкап
```

Calico / сеть нод

NODE проверка оверлея

```
ip link show | egrep 'vxlan.calico|cali|tunl0'
kubectl get ippools.crd.projectcalico.org -o yaml | egrep 'vxlanMode|ipipMode'
ip route get <pod-ip> # dev vxlan.calico = cross-node, dev caliXXX = та же нода
```

kubespray

LOCAL из папки kubespray, .venv активирован

```
ansible all -i inventory/bm-cluster/hosts.yaml -m ping --private-key ~/.ssh/bm_k8s -u root
ansible-playbook -i inventory/bm-cluster/hosts.yaml --private-key ~/.ssh/bm_k8s -u root --become
ansible-playbook ... scale.yaml # добавить ноды
ansible-playbook ... reset.yaml # снести кластер (осторожно!)
```



Спроси нейронку правильно — по темам

AI не заменяет практику — он усиливает. Правило: проси **объяснить** → **дать команды для проверки** → **задать тебе вопрос**. Тогда не получишь галлюцинацию и реально проверишь себя на своём кластере.



ПОСЛЕ DOCKER / СЕТИ

Я поднял docker compose с app + postgres + netshoot.
Объясни путь пакета от app до postgres:
DNS → IP → route → eth0 → veth → bridge. Без «магии».
После объяснения дай 5 команд, которыми я докажу каждый шаг.

↳ **проверь:** `getent hosts postgres · ip route get <ip> · cat /etc/resolv.conf`



ПОСЛЕ KUBE-PROXY / SERVICE

Объясни, как запрос к Kubernetes Service попадает в Pod.
Раздели ответственность: DNS/CoreDNS · ClusterIP · kube-proxy ·
iptables/IPVS · endpoints/endpointslices · CNI/Calico.
Дай команды kubectl/ip/iptables, чтобы проверить путь.

↳ **проверь:** `kubectl get endpointslice · iptables -t nat -L KUBE-SERVICES`



ПОСЛЕ CALICO

У меня kubespray-кластер на 3 VM с Calico.
Помоги понять, какой режим включён: VXLAN, IPIN или BGP.
Дай команды проверки и объясни, как читать вывод.

↳ **проверь:** `kubectl get ippools -o yaml | egrep 'vxlanModelipMode'`



ПОСЛЕ RBAC

Я создал ServiceAccount dev-reader, Role pod-reader и RoleBinding.
Проверь, правильно ли понимаю: authentication = «кто я»,
authorization = «что мне можно», admission = может изменить/запретить.
Задай мне 5 вопросов как на собеседовании.

↳ **проверь:** `kubectl auth can-i get pods -as=system:serviceaccount:dev:dev-reader -n dev`



За этим гайдом стоит **BoostMentor**

Я Виктор — DevOps-инженер и ментор. Веду людей в DevOps на реальной практике: поднимаешь боевые кластеры, ломаешь, чинишь, разбираешь как всё устроено внутри. Эта тетрадь — один блок из серии. Зашло — забирай продолжение.



Telegram-канал — t.me/boostmentor

разборы и следующие блоки серии. Забирай и повторяй руками



Исходники на GitHub — github.com/boost-mentor/ultimate-devops-guide

приложение, все манифесты и лаборатория — клонируешь, поднимаешь, повторяешь каждую тему руками



YouTube

видео-разбор этого блока + собесные ловушки в шортсах



boostmentor.ru

менторство и обучение DevOps — от нуля до оффера, с лабами и инфраструктурой